



西安电子科技大学
XIDIAN UNIVERSITY

NSS-HQC

Yulong Shen, Xutong Mu, Shuaiping Dong, Zidong Zhou, Ahmat Bahadir,
Junbo Ni, Haoyu Zhang, Jinxuan Cui, Ke Cheng, Anxiao Song, Zhenxing Niu,
Lele Zheng

Organizations: Xidian University

April 27, 2026

Abstract

NSS-HQC is a bandwidth-optimized variant of the Hamming Quasi-Cyclic (HQC) key encapsulation mechanism. It integrates two independently studied optimization techniques into a single coherent design: ciphertext compression via Hamming-metric quantization and asymmetric error weight rebalancing. A unified decryption failure rate (DFR) analysis framework enables the joint optimization of both techniques. Concretely, NSS-HQC reduces the ciphertext size by 16–30% relative to the unquantized NSS-HQC baseline at the same parameters, across four security levels, while maintaining the required DFR bound.

This report presents a full written specification of NSS-HQC, including parameters, hashing and serialization rules, key generation, encapsulation, decapsulation, a formal correctness proof, a security analysis reducing IND-CCA2 security to HQC-style quasi-cyclic decoding indistinguishability assumptions, selected parameter families for classical security strengths of 128, 256, 384, and 512 bits (with quantum security strengths of at least 80, 128, 192, and 256 bits respectively), a detailed security strength analysis with ISD cost verification, and implementation-cost estimates.

Contents

1	Introduction	4
1.1	Background	4
1.2	Related Work	4
1.3	Design Rationale	5
2	The NSS-HQC Protocol Specification	6
2.1	Written Specification	6
2.2	Parameters	6
2.3	Preliminaries and Notation	6
2.4	Hashing and Randomness Expansion	7
2.4.1	Hash Functions	7
2.4.2	Randomness Expansion	7
2.4.3	Seed-Based Sampling	7
2.5	Data Types and Conversions	7
2.5.1	Byte Strings	7
2.5.2	Ring Elements	7
2.5.3	Ciphertext Encoding	8
2.5.4	Public-Key and Secret-Key Serialization	8
2.5.5	Correctness Requirement	8
2.6	The Concatenated Error-Correcting Code	8
2.6.1	Internal Code: Duplicated Reed–Muller	8
2.6.2	External Code: Shortened Reed–Solomon	10
2.6.3	Reliability-Based Outer Decoding	11
2.7	The Quantization Module	13
2.7.1	Compression Map	13
2.7.2	Decompression Map	13
2.7.3	Dithering	14
2.8	NSS-HQC-PKE	15
2.8.1	Key Generation Algorithm	16
2.8.2	Encrypt Algorithm	17
2.8.3	Decrypt Algorithm	19
2.9	NSS-HQC-KEM	21
2.9.1	KEM Key Generation	21
2.9.2	Encapsulation Algorithm	21
2.9.3	Decapsulation Algorithm	22
2.10	Correctness	23
2.11	Parameter Sets	25
3	Unified Decryption Failure Rate Analysis	26
3.1	Composite Error Model	26
3.2	Structural Error Distribution	27
3.3	Quantization Error Distribution	27
3.4	Combined Error Probability	28
3.5	DFR Upper Bound	28

3.6	Joint Optimization Procedure	29
4	Security Analysis	29
4.1	Hard Problems Underlying NSS-HQC	29
4.2	Security Proof	31
4.2.1	IND-CPA Security of NSS-HQC-PKE	31
4.2.2	From IND-CPA to IND-CCA2 via the FO Transform	33
4.3	Discussion of the Advantage Loss	34
4.4	Analysis of Known Attacks	35
4.4.1	Key-Recovery Attacks (targeting DQCSD)	35
4.4.2	Message-Recovery Attacks (targeting ciphertext)	35
4.4.3	Decryption Failure Attacks	36
4.4.4	Seed and Target Attacks	36
4.4.5	Asymmetric Weight Security	36
4.5	Security Strength Analysis	37
4.5.1	Methodology	37
4.5.2	ISD Cost Estimation	37
4.5.3	Per-Parameter-Set Analysis	38
4.5.4	Compliance Summary	39
4.5.5	Other Security Properties	39
5	Detailed Performance Analysis	40
5.1	Performance Analysis Methodology	40
5.2	Implementation-Level Operation Profile	41
5.3	Reference Implementation: All Security Levels	42
5.4	Portable Implementation and Optimization Opportunities	43
5.5	Transmission and Storage Overhead	43
5.5.1	Public Key Size	43
5.5.2	Secret Key Size	43
5.5.3	Ciphertext Size	44
5.5.4	Detailed Size Computation	44
5.6	Complete Size Comparison	45
5.7	Comparison with Standard HQC	45
5.7.1	Runtime Comparison on the Benchmark Platform	46
5.8	Memory Footprint Considerations	46
5.9	Summary of Performance Characteristics	46
6	Feature Claims	47
6.1	Innovation	47
6.2	Simplicity	47
6.3	Flexibility	48
6.4	Compatibility	49
6.5	Scalability	50
6.6	Comparison with Other HQC Improvement Proposals	50
6.7	Summary	51

1 Introduction

1.1 Background

The Hamming Quasi-Cyclic (HQC) cryptosystem was selected by NIST on March 11, 2025 as the fifth post-quantum asymmetric encryption algorithm, serving as a backup key encapsulation mechanism alongside the lattice-based ML-KEM (FIPS 203) [1]. The security of HQC rests on the hardness of the quasi-cyclic syndrome decoding (QCSD) problem, a coding-theoretic assumption with roots in the 1978 McEliece cryptosystem. The QCSD problem has been proven to be NP-complete [2], providing a solid theoretical foundation for HQC’s security. In the fourth round of the NIST Post-Quantum Cryptography Standardization process, HQC distinguished itself through its sound cryptographic design and solid reliance on coding-theoretic hard problems, emerging as the sole backup key encapsulation mechanism selected, and is expected to be published as a FIPS standard by 2027.

In HQC, the public key contains a quasi-cyclic parity-check vector h and a syndrome $s = x + h \cdot y$, where (x, y) are sparse secret vectors of Hamming weight w_{sk} in the polynomial ring $R = \mathbb{F}_2[X]/(X^n - 1)$. Encryption masks a message m by encoding it with a public concatenated error-correcting code C and adding structured noise:

$$u = r_1 + h \cdot r_2, \quad v = C.\text{Encode}(m) + s \cdot r_2 + e, \quad (1)$$

where r_1, r_2, e are sparse ephemeral vectors. The ciphertext is the pair (u, v) . Decryption computes $v - u \cdot y = C.\text{Encode}(m) + (x \cdot r_2 + r_1 \cdot y + e)$ and applies the decoder of C to recover m .

The correctness of HQC relies on the decoding capability of C , which must correct the composite error $e' = x \cdot r_2 + r_1 \cdot y + e$. The decryption failure rate (DFR) must be bounded tightly enough for the Fujisaki–Okamoto transform; the submitted NSS-HQC parameter table establishes a baseline correctness margin of $\text{DFR} < 2^{-128}$, while larger claimed CCA proof margins require correspondingly smaller DFR bounds.

HQC has a sound cryptographic design. However, its public-key and ciphertext sizes are substantially larger than those of ML-KEM. For HQC-128, the combined bandwidth is approximately 6.7 KB, compared to roughly 2.3 KB for ML-KEM-768. Reducing this overhead has been a central research objective.

1.2 Related Work

Ciphertext compression. Bitzer et al. (ePrint 2025/2259, EUROCRYPT 2026) [3] introduced the first ciphertext compression technique for code-based cryptography in the Hamming metric, reducing ciphertext size by approximately 4–6%.

Asymmetric error rebalancing. Guan et al. [4] (ePrint 2026/461) observed that the three ephemeral weights (w_{r_1}, w_{r_2}, w_e) contribute differently to the composite error weight. By using asymmetric weights, the expected weight of the composite error is reduced, yielding 10–12% bandwidth savings and approximately 6–8% runtime improvement.

Other related work. Wang et al. [5] proposed HARE (distance-informed erasure decoding, achieving 12–13% combined bandwidth reduction). Bitzer et al. [6] proposed generalized concatenated codes with reliability-based decoding, further improving the DFR–bandwidth trade-off. Cai and Zhang [7] proposed a hardware-friendly decryption scheme based on generalized minimum-distance (GMD) RS decoders. Baldi et al. [8] proposed error-structure-aware decoding strategies beyond the BSC model. Nugier and Deneuville [9] studied improved code

constructions for HQC. Additionally, Annechini et al. [10] eliminated the independence assumption in conventional DFR analysis, deriving a more precise closed-form DFR model that confirms the conservatism of existing approximations. NSS-HQC is complementary to all of the above approaches.

1.3 Design Rationale

NSS-HQC starts from the observation that ciphertext compression and asymmetric rebalancing act on different aspects of HQC and can therefore be composed. The coupling mechanism is captured by the BSC cascade formula:

$$p_{\text{combined}} = p_{\text{struct}} + p_{\text{quant}} - 2 \cdot p_{\text{struct}} \cdot p_{\text{quant}}. \quad (2)$$

A lower p_{struct} (from rebalancing) permits a higher p_{quant} (from more aggressive compression) while still meeting the DFR bound. NSS-HQC integrates both modifications into a three-stage design:

1. Asymmetric noise generation: ephemeral vectors are sampled with rebalanced weights.
2. Encryption with standard HQC algebra, preserving the ring structure and public key format.
3. Ciphertext quantization with dithering before transmission.

The public key format is unchanged. Modifications affect only encapsulation and decapsulation.

Remark 1 (Core modifications relative to the NIST-standardized HQC). *The submission requirements stipulate that algorithms already undergoing international standardization are not accepted unless they involve core modifications. NSS-HQC introduces three core algorithmic modifications to the NIST-standardized HQC that fundamentally alter the encryption and decryption procedures:*

1. **Asymmetric error weight allocation:** *The ephemeral noise distribution is redesigned with non-symmetric weights (w_{r_1}, w_{r_2}, w_e), changing the fundamental error model from a single-parameter BSC to a multi-parameter composite channel. This modification changes the core encryption equation (Eq. 1) and requires a new DFR analysis framework (Section 3).*
2. **Ciphertext quantization with dithered compression:** *A new quantization module is introduced that compresses the ciphertext component v before transmission, fundamentally changing the ciphertext format, the decryption procedure (which now includes decompression), and the error model (which now includes quantization error).*
3. **Reliability-based errors-and-erasures decoding:** *The inner RM decoder is augmented with soft-information output, and the outer RS decoder is upgraded from hard-decision to errors-and-erasures decoding, changing the core decoding pipeline.*

These three modifications interact through the unified DFR framework of Section 3, which is itself a new theoretical contribution. The resulting scheme has different ciphertext format, different error distribution, different decoding algorithm, and different parameter constraints compared to the NIST-standardized HQC.

2 The NSS-HQC Protocol Specification

2.1 Written Specification

This section presents the formal protocol specification of NSS-HQC. The design principle is the joint integration of ciphertext compression and asymmetric error rebalancing into a single KEM specification with a unified correctness and security analysis.

2.2 Parameters

The NSS-HQC KEM is parameterized by:

- n : the code length (a prime with $\text{ord}_n(2) = n - 1$). All operations in $R = \mathbb{F}_2[X]/(X^n - 1)$.
- w_{sk} : Hamming weight of each secret-key polynomial x, y .
- w_{r_1}, w_{r_2}, w_e : weights of the three ephemeral vectors (asymmetric).
- n_1, k_1 : RS code parameters over $\mathbb{F}_{256}$; $\delta_1 = n_1 - k_1 + 1$.
- $n_2 = 128$: RM(1,7) block length; mult: duplication factor.
- $n_C = n_1 \cdot n_2 \cdot \text{mult}$: total code length.
- Q : quantization alphabet size; $b = \log_2(Q)$.
- κ : shared secret length, set to $\max(256, \text{classical_target})$.
- $\lambda_{\text{salt}}, \lambda_{\text{seed}}$: salt and seed lengths in bits.

2.3 Preliminaries and Notation

If X is a finite set, $x \xleftarrow{\$} X$ denotes uniform random sampling. If A is a probabilistic algorithm, $y \leftarrow A(x)$ denotes that y is the output of A on input x .

Polynomial ring. The ambient algebraic structure in NSS-HQC is the quotient ring

$$R = \mathbb{F}_2[X]/(X^n - 1), \quad (3)$$

where n is a prime satisfying $\text{ord}_n(2) = n - 1$ (i.e., 2 is a primitive root modulo n). This condition ensures that $X^n - 1$ has only two irreducible factors over $\mathbb{F}_2$: namely $(X - 1)$ and a single irreducible polynomial of degree $n - 1$, which guarantees that the ring R has a “quasi-cyclic” structure with favorable algebraic properties for both the security assumption and efficient polynomial multiplication.

Elements of R are represented as binary polynomials of degree at most $n - 1$, equivalently as binary vectors of length n . Addition in R is componentwise XOR:

$$(a_0, a_1, \dots, a_{n-1}) + (b_0, b_1, \dots, b_{n-1}) = (a_0 \oplus b_0, a_1 \oplus b_1, \dots, a_{n-1} \oplus b_{n-1}). \quad (4)$$

Multiplication in R is polynomial multiplication modulo $X^n - 1$, which is equivalent to cyclic convolution. For $a, b \in R$, the product $c = a \cdot b$ has coefficients

$$c_k = \bigoplus_{i+j \equiv k \pmod{n}} a_i \cdot b_j, \quad k = 0, 1, \dots, n-1. \quad (5)$$

This operation can be implemented efficiently in $O(n \log n \log \log n)$ time using number-theoretic transforms over suitable extension fields.

Hamming weight. For $v \in \mathbb{F}_2^n$, the Hamming weight $\text{wt}(v) = |\{i : v_i \neq 0\}|$ counts the number of nonzero coordinates.

Fixed-weight sampling. We write $v \xleftarrow{\$} R_w$ to denote that v is sampled uniformly at random from the set

$$R_w = \{v \in R : \text{wt}(v) = w\}, \quad (6)$$

i.e., the set of all elements of R with exactly w nonzero coefficients. This sampling is performed using constant-time Fisher–Yates shuffling to avoid timing side channels.

Concatenated code. The error-correcting code C is the concatenation of an external shortened Reed–Solomon code over $\mathbb{F}_{256}$ and an internal duplicated first-order Reed–Muller code. We write $C.\text{Encode} : \{0, 1\}^{8k_1} \rightarrow \mathbb{F}_2^{n_C}$ for the encoding map and $C.\text{Decode} : \mathbb{F}_2^{n_C} \rightarrow \{0, 1\}^{8k_1} \cup \{\perp\}$ for the decoding map, where $\perp$ denotes a decoding failure.

Codeword projection. Ring elements have length n , whereas the encoded payload has length $n_C = n_1 \cdot n_2 \cdot \text{mult} \leq n$. We therefore use the projection

$$\pi_{n_C} : R \rightarrow \mathbb{F}_2^{n_C}, \quad \pi_{n_C}(a) = (a_0, \dots, a_{n_C-1}) \quad (7)$$

for every ring expression that is added to, or subtracted from, a codeword of C . Equivalently, the PKE layer uses the first n_C coordinates of each ring product as the codeword support.

Quantization module. The quantization module is denoted Q and consists of two block-wise operations: $Q.\text{Compress} : \mathbb{F}_2^{\text{mult}} \rightarrow \{0, \dots, Q-1\}$ maps a binary block of length mult to a quantized symbol, and $Q.\text{Decompress} : \{0, \dots, Q-1\} \rightarrow \mathbb{F}_2^{\text{mult}}$ maps a quantized symbol back to a binary block.

2.4 Hashing and Randomness Expansion

2.4.1 Hash Functions

$G : \{0, 1\}^* \rightarrow \{0, 1\}^{512}$ (SHA3-512); $H_K : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$ (SHAKE256); $I : \{0, 1\}^* \rightarrow \{0, 1\}^{512}$ (SHA3-512). Each uses domain separation per [11].

2.4.2 Randomness Expansion

$\text{Expand}(\text{seed}, \ell) \in \{0, 1\}^\ell$ is a deterministic XOF-based expansion via SHAKE256.

2.4.3 Seed-Based Sampling

All ephemeral vectors are derived from a seed θ via constant-time rejection sampling. The weight triple (w_{r_1}, w_{r_2}, w_e) is read from the NSS-HQC parameter set.

2.5 Data Types and Conversions

2.5.1 Byte Strings

All external interfaces use byte strings. Concatenation is written $\|$.

2.5.2 Ring Elements

An element of R is n bits packed into $\lceil n/8 \rceil$ bytes in little-endian order.

2.5.3 Ciphertext Encoding

In NSS-HQC, the ciphertext replaces v with $\hat{v}$, requiring $n_1 \cdot n_2 \cdot b$ bits:

$$\text{ct} = \text{Encode}(u) \parallel \text{Encode}(\hat{v}). \quad (8)$$

Here u is encoded as an n -bit ring element in little-endian order. The quantized symbols $\hat{v}_0, \dots, \hat{v}_{n_1 n_2 - 1}$ are serialized in increasing block index; each symbol is represented by exactly $b = \log_2 Q$ bits in little-endian bit order, and any unused high bits of the final byte are set to zero. This serialization rule is part of the PKE algorithm, because the FO re-encryption check compares ciphertext byte strings.

2.5.4 Public-Key and Secret-Key Serialization

$\text{pk} = \text{seed}_{\text{PKE}} \parallel \text{Encode}(s)$. $\text{sk} = (\text{seed}_{\text{sk}}, \sigma, \text{pk})$ where $\sigma \in \{0, 1\}^\kappa$ and $\text{seed}_{\text{sk}} \in \{0, 1\}^{\lambda_{\text{seed}}}$. The sparse vectors (x, y) are regenerated from seed_{sk} only when needed; PKE decryption expands y only, while x is needed for key generation and for the correctness analysis.

2.5.5 Correctness Requirement

$\text{Decode}(\text{Encode}(x)) = x$ for all x in the domain.

2.6 The Concatenated Error-Correcting Code

The error-correcting code C used in NSS-HQC is a two-level concatenated code, following the same structural design as the standard HQC specification [11]. The purpose of this code is to protect the plaintext message m against the composite error e'' introduced during encryption and quantization, ensuring that the decryption failure rate remains below the required threshold of 2^{-128} .

2.6.1 Internal Code: Duplicated Reed–Muller

The internal (inner) code is a duplicated first-order Reed–Muller code. The base code is $\text{RM}(1, 7)$, a $[128, 8, 64]$ binary linear code. Its generator matrix consists of the 8×128 matrix whose rows are the constant-1 vector and the seven coordinate functions of $\mathbb{F}_2^7$. The minimum distance $d_{\text{RM}} = 64 = n_2/2$ is half the block length, which provides strong error correction.

To increase the effective minimum distance, the RM code is duplicated by a factor of mult : each codeword bit is repeated mult times, yielding a $[128 \cdot \text{mult}, 8, 64 \cdot \text{mult}]$ code. The encoding of a single $\mathbb{F}_{256}$ symbol $\sigma = (\sigma_0, \dots, \sigma_7) \in \mathbb{F}_2^8$ is:

$$\text{RM.Encode}(\sigma) = \text{dup}_{\text{mult}}(\text{RM}_{(1,7)}. \text{Encode}(\sigma)) \in \mathbb{F}_2^{128 \cdot \text{mult}}, \quad (9)$$

where $\text{dup}_{\text{mult}}(c_0, c_1, \dots, c_{127}) = (\underbrace{c_0, \dots, c_0}_{\text{mult}}, \underbrace{c_1, \dots, c_1}_{\text{mult}}, \dots, \underbrace{c_{127}, \dots, c_{127}}_{\text{mult}})$.

Decoding via fast Hadamard transform (FHT). Decoding of the duplicated RM code uses a soft-combined FHT over the whole duplicated block; the mult copies are not first collapsed by a hard majority vote. For a received block

$$r = (r_{i,\ell})_{0 \leq i < 128, 0 \leq \ell < \text{mult}} \in \{0, 1\}^{128 \cdot \text{mult}},$$

the decoder first accumulates one integer soft value per RM coordinate,

$$z_i = \sum_{\ell=0}^{\text{mult}-1} (-1)^{r_{i,\ell}} \in \{-\text{mult}, -\text{mult}+2, \dots, \text{mult}\}.$$

The fast Hadamard transform (FHT) computes $2^7 = 128$ Walsh–Hadamard correlations indexed by the 7-bit linear part $a \in \mathbb{F}_2^7$:

$$\hat{r}_a = \sum_{i=0}^{127} z_i \cdot (-1)^{\langle a, i \rangle} = \sum_{i=0}^{127} \sum_{\ell=0}^{\text{mult}-1} (-1)^{r_{i,\ell}} \cdot (-1)^{\langle a, i \rangle}, \quad a \in \mathbb{F}_2^7, \quad (10)$$

where $\langle a, i \rangle$ denotes the inner product over $\mathbb{F}_2$ of the 7-bit vectors a and i (identifying $\{0, \dots, 127\}$ with $\mathbb{F}_2^7$). The decoded linear part and affine bit are recovered as:

$$a^* = \arg \max_{a \in \mathbb{F}_2^7} |\hat{r}_a|, \quad (11)$$

$$b^* = \mathbf{1}[\hat{r}_{a^*} < 0], \quad (12)$$

and the decoded 8-bit symbol is $s_{\text{decoded}} = (b^* \| a^*) \in \{0, 1\}^8$. Under the independent BSC model on the duplicated coordinates, this is the maximum-likelihood (ML) decision rule for the duplicated RM(1,7) block. The implementation requires one linear accumulation pass over $128 \cdot \text{mult}$ received bits plus a 128-point FHT (half the cost of a flat 256-point implementation).

Reliability output. In addition to the decoded symbol, the FHT decoder produces a reliability measure for each block. NSS-HQC adopts the **normalized maximum FHT amplitude** as the reliability metric. Concretely, for the i -th inner block, after computing all 128 FHT coefficients $\{\hat{r}_a\}_{a \in \mathbb{F}_2^7}$ and selecting the decoded linear part $a^* = \arg \max_a |\hat{r}_a|$, the reliability is defined as:

$$\rho_i = \frac{|\hat{r}_{a^*}|}{128 \cdot \text{mult}} = \frac{\max_{a \in \mathbb{F}_2^7} |\hat{r}_a|}{128 \cdot \text{mult}}. \quad (13)$$

The denominator $128 \cdot \text{mult}$ is the maximum possible FHT amplitude (achieved when the received block is a perfect codeword with no errors), so $\rho_i \in [0, 1]$.

Interpretation. The value $\rho_i = 1$ indicates that the received block matched a codeword exactly (zero errors in the block). A value $\rho_i \approx 0$ indicates that no codeword correlates significantly with the received block, meaning the error density is near 50% (essentially random). Intermediate values indicate partial corruption. The reliability ρ_i is a monotonically decreasing function of the number of bit errors within block i : as the number of errors increases, the FHT peak amplitude decreases.

Remark 2 (Choice of reliability metric). *Three common reliability metrics for FHT-based RM decoding have been considered in the literature:*

- (a) *Maximum amplitude:* $\rho = \max_{a \in \mathbb{F}_2^7} |\hat{r}_a| / (128 \cdot \text{mult})$. This is the metric adopted by NSS-HQC (Eq. 13).
- (b) *Gap metric:* $\rho = (\max_1 - \max_2) / (128 \cdot \text{mult})$, where $\max_1$ and $\max_2$ are the two largest FHT amplitudes. This is more robust near the decoding boundary but requires sorting.
- (c) *Log-likelihood ratio:* $\rho = \log(P_{\text{correct}} / P_{\text{incorrect}})$. Information-theoretically optimal but computationally expensive.

NSS-HQC uses metric (a) for simplicity and efficiency: it requires no additional computation beyond the FHT that is already performed for decoding, and it provides sufficient discrimination for the erasure threshold mechanism. The gap metric (b) may be considered in future optimized implementations.

2.6.2 External Code: Shortened Reed–Solomon

The external (outer) code is a shortened Reed–Solomon code $\text{RS}[n_1, k_1, \delta_1]$ over $\mathbb{F}_{256}$, where $\delta_1 = n_1 - k_1 + 1$ is the minimum distance. The RS code is defined in **evaluation form**: it encodes k_1 message symbols $m = (m_0, m_1, \dots, m_{k_1-1}) \in \mathbb{F}_{256}^{k_1}$ into n_1 codeword symbols $(c_0, c_1, \dots, c_{n_1-1}) \in \mathbb{F}_{256}^{n_1}$ by evaluating the message polynomial

$$M(X) = \sum_{i=0}^{k_1-1} m_i X^i \in \mathbb{F}_{256}[X] \quad (14)$$

at n_1 distinct evaluation points $(\alpha^0, \alpha^1, \dots, \alpha^{n_1-1})$ in $\mathbb{F}_{256}$, where α is a primitive element of $\mathbb{F}_{256}$. That is:

$$c_j = M(\alpha^j) = \sum_{i=0}^{k_1-1} m_i \cdot \alpha^{ji}, \quad j = 0, 1, \dots, n_1 - 1. \quad (15)$$

Since $|\mathbb{F}_{256}| = 256 \geq n_1$ for all parameter sets (see Remark 3), all evaluation points are distinct and the code is well-defined.

Galois field specification. The field $\mathbb{F}_{256}$ is constructed as $\mathbb{F}_2[X]/(X^8 + X^4 + X^3 + X^2 + 1)$. This is the same irreducible polynomial used in the standard HQC specification [11]. The primitive element α is the equivalence class of X in this quotient field. Each byte value $b \in \{0, 1, \dots, 255\}$ is identified with the field element $\sum_{i=0}^7 b_i \alpha^i$ where $(b_0, b_1, \dots, b_7)$ is the binary representation of b (least significant bit first).

Message byte ordering. The message bytes $(m_0, m_1, \dots, m_{k_1-1})$ are mapped to polynomial coefficients in **low-degree-first** order: m_0 is the constant term and m_{k_1-1} is the coefficient of X^{k_1-1} .

Syndrome definition. For a received word $\mathbf{r} = (r_0, r_1, \dots, r_{n_1-1}) \in \mathbb{F}_{256}^{n_1}$, the syndrome values used in the Berlekamp–Massey decoder (Section 2.6.3) are defined as:

$$S_j = \sum_{i=0}^{n_1-1} r_i \cdot \alpha^{ij}, \quad j = 1, 2, \dots, n_1 - k_1. \quad (16)$$

This definition is consistent with the evaluation-form encoding above: if $\mathbf{r}$ is a valid codeword (i.e., $r_j = M(\alpha^j)$ for some degree- $(k_1 - 1)$ polynomial M), then all syndromes are zero. The Berlekamp–Massey algorithm and Forney’s formula (Eq. 22) in Section 2.6.3 operate on these syndromes and are fully compatible with the evaluation-form encoding.

The RS code can correct up to $t = \lfloor (\delta_1 - 1)/2 \rfloor$ symbol errors using hard-decision decoding, or a combination of t' errors and ϵ erasures satisfying $2t' + \epsilon \leq \delta_1 - 1$ using errors-and-erasures decoding (Section 2.6.3).

Remark 3 (RS code length constraint). *A Reed–Solomon code over $\mathbb{F}_{256}$ has maximum block length $n_1^{\max} = 255$ (or 256 with the point at infinity). All four NSS-HQC parameter sets satisfy $n_1 \leq 130 < 255$, so a single RS code over $\mathbb{F}_{256}$ is sufficient in all cases. No extension*

to larger Galois fields (such as $\mathbb{F}_{2^{16}}$) or parallel RS constructions is required. This is an important implementation consideration: the entire RS encoder and decoder operates in $\mathbb{F}_{256} = \mathbb{F}_2[X]/(X^8 + X^4 + X^3 + X^2 + 1)$ using standard 8-bit Galois field arithmetic for all four security levels.

Remark 4 (Clarification on Table 1 column ordering). In Table 1, the column labeled w_e precedes the column labeled n_1 . For NSS-HQC-512, $w_e = 372$ and $n_1 = 130$. The value 372 is the additive error weight, not the RS code length. The RS code length $n_1 = 130$ is well within the $\mathbb{F}_{256}$ limit of 255.

Full concatenated encoding. The complete encoding map $C.\text{Encode} : \{0, 1\}^{8k_1} \rightarrow \mathbb{F}_2^{n_C}$ operates as follows. Given a message $m \in \{0, 1\}^{8k_1}$:

1. Parse m as k_1 bytes, each representing an element of $\mathbb{F}_{256}$: $m \mapsto (m_0, m_1, \dots, m_{k_1-1})$.
2. Apply RS encoding: $(c_0, c_1, \dots, c_{n_1-1}) \leftarrow \text{RS.Encode}(m_0, \dots, m_{k_1-1})$.
3. For each RS symbol c_i , apply the duplicated RM encoding: $w_i \leftarrow \text{RM.Encode}(c_i) \in \mathbb{F}_2^{128 \cdot \text{mult}}$.
4. Concatenate: $C.\text{Encode}(m) = w_0 \parallel w_1 \parallel \dots \parallel w_{n_1-1} \in \mathbb{F}_2^{n_C}$.

The total codeword length is $n_C = n_1 \cdot n_2 \cdot \text{mult} = n_1 \cdot 128 \cdot \text{mult}$.

2.6.3 Reliability-Based Outer Decoding

NSS-HQC uses the reliability information from the inner RM decoder (Eq. 13) to improve the outer RS decoding beyond hard-decision-only performance. After inner decoding of all n_1 blocks, each RS symbol s_i has an associated reliability $\rho_i \in [0, 1]$. Symbols whose reliability falls below a threshold τ are deemed unreliable and are declared as *erasures* rather than being treated as potentially correct symbols:

$$\text{erasure_mask} = \{i \in \{1, \dots, n_1\} : \rho_i < \tau\}. \quad (17)$$

Let $\epsilon = |\text{erasure_mask}|$ denote the number of declared erasures. The RS decoder then performs *errors-and-erasures* decoding, which can simultaneously correct t' errors and ϵ erasures provided that

$$2t' + \epsilon \leq \delta_1 - 1 = n_1 - k_1. \quad (18)$$

This is strictly more powerful than pure error correction (which requires $2t \leq \delta_1 - 1$), because each erasure “costs” only one unit of redundancy instead of two. The net effect is that the RS decoder can tolerate more total failures at the inner code level, which in turn allows shorter code parameters (smaller n_1) at the same DFR.

Concrete decoding algorithm. NSS-HQC specifies the **Berlekamp–Massey algorithm with Forney’s formula** as the RS errors-and-erasures decoder. The decoding procedure consists of five steps:

1. **Syndrome computation.** Compute $2t_{\max} = n_1 - k_1$ syndromes from the received word $\mathbf{r} = (s_1, \dots, s_{n_1}) \in \mathbb{F}_{256}^{n_1}$:

$$S_j = \sum_{i=0}^{n_1-1} r_i \cdot \alpha^{ij}, \quad j = 1, 2, \dots, n_1 - k_1, \quad (19)$$

where α is a primitive element of $\mathbb{F}_{256}$. The purpose of syndrome computation is to extract error-dependent information from the received word; if $\mathbf{r}$ is a valid codeword, all syndromes are zero.

2. **Erasure locator polynomial.** From the erasure mask, construct the erasure locator polynomial:

$$\Gamma(x) = \prod_{i \in \text{erasure_mask}} (1 - \alpha^i x). \quad (20)$$

This polynomial has degree ϵ and its roots identify the positions of the declared erasures. Its purpose is to “inform” the key equation solver about the known error locations.

3. **Modified syndromes.** Compute the modified syndrome polynomial $T(x) = S(x) \cdot \Gamma(x) \pmod{x^{n_1-k_1}}$, where $S(x) = \sum_{j=1}^{n_1-k_1} S_j x^{j-1}$. The modified syndromes account for the contribution of the known erasure positions, leaving only the unknown error positions to be determined.
4. **Error locator polynomial via Berlekamp–Massey.** Apply the Berlekamp–Massey algorithm to the modified syndromes to find the error locator polynomial $\sigma(x)$:

$$\sigma(x) = 1 + \sigma_1 x + \sigma_2 x^2 + \cdots + \sigma_{t'} x^{t'}, \quad (21)$$

where t' is the number of errors (distinct from erasures). The Berlekamp–Massey algorithm runs in $O((n_1 - k_1)^2)$ operations over $\mathbb{F}_{256}$. The roots of $\sigma(x)$ identify the error positions. **Decoding failure condition:** if $\deg(\sigma) > \lfloor (n_1 - k_1 - \epsilon)/2 \rfloor$, the decoder declares failure and returns $\perp$.

5. **Error values via Forney’s formula.** Once the error positions are identified (via Chien search on $\sigma(x)$), the error magnitudes at each position i are computed using Forney’s formula:

$$e_i = -\frac{\Omega(\alpha^{-i})}{\Lambda'(\alpha^{-i})}, \quad (22)$$

where $\Lambda(x) = \sigma(x) \cdot \Gamma(x)$ is the combined error-erasure locator, $\Lambda'(x)$ is its formal derivative, and $\Omega(x) = S(x) \cdot \Lambda(x) \pmod{x^{n_1-k_1}}$ is the error evaluator polynomial. The purpose of Forney’s formula is to compute the magnitude of each error/erasure without solving a full linear system.

Decoding failure detection. The decoder declares failure and returns $\perp$ in the following cases:

1. The Berlekamp–Massey algorithm produces $\deg(\sigma) > \lfloor (n_1 - k_1 - \epsilon)/2 \rfloor$, indicating more errors than the code can correct.
2. The Chien search fails to find all roots of $\sigma(x)$, indicating an inconsistent error pattern.
3. The corrected codeword has nonzero syndromes (post-correction verification), indicating an uncorrectable error pattern.

Any of these conditions causes the PKE decryption to return $\perp$, which triggers implicit rejection at the KEM level.

The threshold τ is a fixed parameter for each security level, specified in Table 4. It is chosen during parameter optimization to minimize the overall DFR. Setting τ too high wastes RS redundancy on false erasures; setting τ too low fails to catch genuinely erroneous symbols.

2.7 The Quantization Module

The quantization module is the mechanism by which the ciphertext component v is compressed before transmission. Its purpose is to reduce the bandwidth required to transmit v by mapping groups of mult binary coordinates to a single symbol from a smaller alphabet $\{0, 1, \dots, Q-1\}$, at the cost of introducing a controlled quantization error $\hat{e}$ that is absorbed by the error-correcting code C .

2.7.1 Compression Map

The compression map operates blockwise. For each group of mult consecutive bits in v , the map computes a single quantized symbol. Given a block $w = (w_0, w_1, \dots, w_{\text{mult}-1}) \in \mathbb{F}_2^{\text{mult}}$, the **block weight** is $\text{bw}(w) = \sum_{i=0}^{\text{mult}-1} w_i$, i.e., the number of 1-bits in the block. The compression map then produces:

$$Q.\text{Compress}(w) = \left\lfloor \frac{2 \text{bw}(w) (Q-1) + \text{mult}}{2 \text{mult}} \right\rfloor \in \{0, 1, \dots, Q-1\}, \quad (23)$$

which is integer rounding to the nearest value, with half-way cases rounded upward. This fixed integer rule avoids floating-point behavior and makes CompressBlockwise byte-for-byte deterministic across implementations. It maps the block weight, which ranges from 0 to mult, into the quantization alphabet $\{0, 1, \dots, Q-1\}$ by uniform partitioning of the weight range.

Example. For mult = 5 and $Q = 4$, the mapping is:

Block weight $\text{bw}(w)$	Scaled value $\text{bw}(w) \cdot 3/5$	$Q.\text{Compress}(w)$
0	0.0	0
1	0.6	1
2	1.2	1
3	1.8	2
4	2.4	2
5	3.0	3

Rounding maps blocks of weight 1 and 2 to the same quantized symbol (1), and weight 3 and 4 to the same symbol (2); this is the controlled quantization error absorbed by the outer RS code.

Blockwise application. The full ciphertext component $v_{\text{raw}} \in \mathbb{F}_2^{n_C}$ is partitioned into $n_1 \cdot n_2$ consecutive blocks of mult bits each (since $n_C = n_1 \cdot n_2 \cdot \text{mult}$). Each block is independently compressed:

$$Q.\text{CompressBlockwise}(v) = (Q.\text{Compress}(v[0 : \text{mult}]), \\ Q.\text{Compress}(v[\text{mult} : 2\text{mult}]), \dots) \in \{0, \dots, Q-1\}^{n_1 \cdot n_2}. \quad (24)$$

The compressed representation $\hat{v}$ requires $n_1 \cdot n_2 \cdot b$ bits, where $b = \log_2 Q$, compared to the original $n_1 \cdot n_2 \cdot \text{mult}$ bits. The compression ratio on the v component is therefore b/mult .

2.7.2 Decompression Map

Given a quantized symbol $c \in \{0, 1, \dots, Q-1\}$, the decompression map reconstructs a binary block of length mult. The decompressed block is a **constant block** — all bits are set to the same

value — determined by the majority decision:

$$b = \left\lfloor \frac{2c + (Q - 1)}{2(Q - 1)} \right\rfloor \in \{0, 1\}, \quad (25)$$

$$Q.\text{Decompress}(c) = (\underbrace{b, b, \dots, b}_{\text{mult}}) \in \mathbb{F}_2^{\text{mult}}. \quad (26)$$

For $Q = 4$ and $\text{mult} = 5$, this gives:

Quantized c	$c/(Q - 1)$	Majority b	Decompressed block
0	0.00	0	(0, 0, 0, 0, 0)
1	0.33	0	(0, 0, 0, 0, 0)
2	0.67	1	(1, 1, 1, 1, 1)
3	1.00	1	(1, 1, 1, 1, 1)

Remark 5 (Rationale for constant-block decompression). *The internal duplicated RM code produces codewords where each group of mult consecutive bits are copies of the same RM bit. The constant-block decompression preserves this duplication structure: the decompressed output is always a valid duplication pattern (all-zero or all-one). A naïve alternative — outputting a prefix-weight block such as (1, 1, 0) for $c = 2$ — would destroy the duplication alignment, causing the subsequent RM decoder to see bit-position artifacts inside a duplication group. This design choice should not be read as claiming that the actual quantization error relative to v_{raw} is always a constant block: once the structural HQC noise is present, the difference between v_{raw} and the reconstructed constant block may contain mixed bit patterns. What is guaranteed by the implemented decompression is that the dequantized value before subtracting $u \cdot y$ is aligned with the RM duplication structure.*

The decompression is deterministic and introduces no additional randomness. For $Q = 4$, the symbols $c = 0$ and $c = 3$ indicate stronger agreement with the hard decisions 0 and 1, while $c = 1$ and $c = 2$ indicate weaker agreement. The reference PKE decryption algorithm below uses only the deterministic hard-decompressed vector. An optimized implementation may pass these symbol-level confidences as additional weights to a soft-input RM decoder, but such an optimization must be analyzed together with the DFR bound. The formal decoding path specified here relies on the reliability metric ρ_i computed by the RM/FHT decoder from the received block w (Eq. 13).

2.7.3 Dithering

A naïve application of quantization to the ciphertext v would produce a quantization error whose distribution depends on the message m (through $C.\text{Encode}(m)$). To ensure that the DFR analysis holds uniformly over all messages, a **deterministic dither vector** is applied before compression and removed after decompression.

Block-level dithering. The dither operates at the level of RM duplication groups, not at the level of individual bits. A short dither vector $d_{\text{base}} \in \mathbb{F}_2^{n_1 \cdot n_2}$ is generated (one bit per RM code bit), and then expanded to the full codeword length by replicating each dither bit mult times:

$$d_{\text{base}} = \text{Expand}(\text{salt} \parallel \text{pk} \parallel \text{"dither"}, n_1 \cdot n_2), \quad (27)$$

$$d = \text{dup}_{\text{mult}}(d_{\text{base}}) = (\underbrace{d_1, \dots, d_1}_{\text{mult}}, \underbrace{d_2, \dots, d_2}_{\text{mult}}, \dots, \underbrace{d_{n_1 n_2}, \dots, d_{n_1 n_2}}_{\text{mult}}) \in \mathbb{F}_2^{n_C}. \quad (28)$$

Within each duplication group, all mult bits receive the *same* dither bit.

Remark 6 (Rationale for block-level dithering). *Block-level dithering is essential for compatibility with the constant-block decompression. Since the decompression map outputs a constant block (b, b, b, b, b) and the dither within each group is also constant (d, d, d, d, d) , the dither removal produces $(b \oplus d, b \oplus d, b \oplus d, b \oplus d, b \oplus d)$, which is again a constant block. This preserves the duplication structure that the RM decoder relies on. By contrast, bit-level dithering (where each of the mult bits in a group receives an independent dither bit) would produce $(b \oplus d_1, b \oplus d_2, b \oplus d_3, b \oplus d_4, b \oplus d_5)$ after dither removal, which is not a constant block and would destroy the duplication alignment, causing approximately 33% additional bit errors per group.*

Key design decision. The dither vector d_{base} is public and deterministic given the salt and the public key:

$$d_{\text{base}} = \text{Expand}(\text{salt} \parallel \text{pk} \parallel \text{"dither"}, n_1 \cdot n_2), \quad (29)$$

and **not** from the encapsulation seed θ or the message m . This avoids a circular dependency in decapsulation: the receiver needs d to decompress the ciphertext *before* recovering m , but θ depends on m . Since salt and pk are known to the receiver, d can be computed before decryption begins. This also keeps the compression layer as a deterministic public post-processing step, which is the property used in the IND-CPA proof.

Usage protocol. During encryption:

$$\hat{v} = Q.\text{CompressBlockwise}(v_{\text{raw}} \oplus d). \quad (30)$$

During decryption:

$$\tilde{v} = Q.\text{DecompressBlockwise}(\hat{v}) \oplus d. \quad (31)$$

Since the dither is constant within each group and the decompression outputs a constant block, the dither removal preserves the duplication structure at the input of the subsequent noise-cancellation step. The quantization error $\hat{e} = \tilde{v} - v_{\text{raw}}$ is therefore treated as the discrepancy introduced by this deterministic compress-decompress layer. In the real algorithm this discrepancy need not be a constant block relative to v_{raw} ; Section 3 uses a conservative hard-decision proxy to bound its effect on the RM decoding layer.

2.8 NSS-HQC-PKE

The underlying public-key encryption scheme NSS-HQC-PKE consists of three algorithms: key generation, encryption, and decryption. We first present the pseudocode for each algorithm, followed by a detailed step-by-step explanation of the mathematical operations and their cryptographic purposes.

2.8.1 Key Generation Algorithm

Algorithm 1. NSS-HQC-PKE.KeyGen(n, w_{sk})

Require: System parameters (n, w_{sk})

Ensure: Public key pk , secret key sk

```

1:  $\text{seed}_{\text{PKE}} \xleftarrow{\$} \{0, 1\}^{\lambda_{\text{seed}}}$ 
2:  $\text{seed}_{\text{sk}} \xleftarrow{\$} \{0, 1\}^{\lambda_{\text{seed}}}$ 
3:  $\sigma \xleftarrow{\$} \{0, 1\}^{\kappa}$ 
4:  $h \leftarrow \text{Expand}(\text{seed}_{\text{PKE}} \parallel \text{"h"}, n)$ 
5:  $(\text{seed}_x, \text{seed}_y) \leftarrow I(\text{seed}_{\text{sk}})$ 
6:  $\triangleright$  SHA3-512 splits one 256-bit seed into two 256-bit sub-seeds
7:  $x \xleftarrow{\$} R_{w_{\text{sk}}}$  using  $\text{Expand}(\text{seed}_x, \cdot)$ 
8:  $y \xleftarrow{\$} R_{w_{\text{sk}}}$  using  $\text{Expand}(\text{seed}_y, \cdot)$ 
9:  $s \leftarrow x + h \cdot y$ 
10:  $\text{pk} \leftarrow (\text{seed}_{\text{PKE}}, s); \text{sk} \leftarrow (\text{seed}_{\text{sk}}, \sigma, \text{pk})$ 
11: return ( $\text{pk}, \text{sk}$ )
```

Detailed explanation of each step:

Step 1: PKE seed sampling. A uniformly random seed $\text{seed}_{\text{PKE}} \in \{0, 1\}^{\lambda_{\text{seed}}}$ is sampled. This seed is used to deterministically derive the public vector h and is stored as part of the public key.

Step 2: Secret key seed sampling. A second uniformly random seed $\text{seed}_{\text{sk}} \in \{0, 1\}^{\lambda_{\text{seed}}}$ is sampled *independently* of seed_{PKE} . This seed is used exclusively to derive the sparse secret key vectors (x, y) on demand during decapsulation, and is the only secret key material that needs to be stored alongside σ and pk . Separating seed_{sk} from seed_{PKE} ensures that the secret key derivation is independent of the public key derivation.

Step 3: Rejection secret. A uniformly random string $\sigma \in \{0, 1\}^{\kappa}$ is sampled independently. This value is used exclusively in the KEM layer for implicit rejection: when decapsulation detects an invalid ciphertext, it uses σ to derive a pseudorandom shared secret rather than returning $\perp$, which prevents chosen-ciphertext attackers from distinguishing valid decapsulations from rejections.

Step 4: Public vector generation. The public quasi-cyclic vector $h \in R$ is generated by expanding seed_{PKE} through the XOF (SHAKE256) with a domain-separation tag "h". This produces a pseudorandom element of R that is computationally indistinguishable from a uniformly random ring element. The public key stores only seed_{PKE} (not the full h), saving $\lceil n/8 \rceil - \lambda_{\text{seed}}/8$ bytes.

Step 5: Sub-seed derivation. The single secret seed seed_{sk} is split into two independent 256-bit sub-seeds $(\text{seed}_x, \text{seed}_y) \leftarrow I(\text{seed}_{\text{sk}})$ via SHA3-512, ensuring that the two secret vectors are derived from independent randomness.

Steps 6–7: Secret key generation. Two sparse vectors $x, y \in R_{w_{\text{sk}}}$ are sampled, each with exactly w_{sk} nonzero coordinates, using seed_x and seed_y respectively (via constant-time rejection sampling). The pair (x, y) constitutes the structured secret key. Note that (x, y) are *not* stored in the secret key; they are regenerated from seed_{sk} on demand during decryption.

Step 8: Syndrome computation. The public syndrome is computed as

$$s = x + h \cdot y \in R. \quad (32)$$

This is the core algebraic relation underlying the DQCSD assumption: given (h, s) , recovering the sparse pair (x, y) is computationally hard. The multiplication $h \cdot y$ is a cyclic convolution in R , computable in $O(n \log n)$ time. The addition is a bitwise XOR.

Step 9: Key output. The public key $\text{pk} = (\text{seed}_{\text{PKE}}, s)$ consists of the compact seed (from which h can be reconstructed) and the syndrome s . The secret key $\text{sk} = (\text{seed}_{\text{sk}}, \sigma, \text{pk})$ stores only the secret seed, the rejection secret, and a copy of the public key. The sparse vectors (x, y) are regenerated on demand during decryption from seed_{sk} , reducing the serialised secret key size by $2\lceil n/8 \rceil$ bytes compared to storing (x, y) explicitly.

2.8.2 Encrypt Algorithm

Algorithm 2. NSS-HQC-PKE.Encrypt($\text{pk}, m, \theta, \text{salt}$)

Require: Public key pk , message $m \in \{0, 1\}^{8k_1}$, randomness θ , salt

Ensure: Ciphertext $\text{ct} = (u, \hat{v})$

- 1: Regenerate h from seed_{PKE}
 - 2: $r_1 \xleftarrow{\$} R_{w_{r_1}}$ using $\text{Expand}(\theta \parallel \text{"r1"}, \cdot)$
 - 3: $\triangleright$ *asymmetric weight*
 - 4: $r_2 \xleftarrow{\$} R_{w_{r_2}}$ using $\text{Expand}(\theta \parallel \text{"r2"}, \cdot)$
 - 5: $e \xleftarrow{\$} R_{w_e}$ using $\text{Expand}(\theta \parallel \text{"e"}, \cdot)$
 - 6: $u \leftarrow r_1 + h \cdot r_2$
 - 7: $v_{\text{raw}} \leftarrow C.\text{Encode}(m) + \pi_{n_C}(s \cdot r_2 + e)$
 - 8: $d_{\text{base}} \leftarrow \text{Expand}(\text{salt} \parallel \text{pk} \parallel \text{"dither"}, n_1 \cdot n_2)$; $d \leftarrow \text{dup}_{\text{mult}}(d_{\text{base}})$
 - 9: $\triangleright$ *block-level dither*
 - 10: $\hat{v} \leftarrow Q.\text{CompressBlockwise}(v_{\text{raw}} \oplus d)$
 - 11: **return** $\text{ct} = (u, \hat{v})$
-

Detailed explanation of each step:

Step 1: Public vector reconstruction. The public vector $h \in R$ is regenerated from the seed stored in the public key: $h \leftarrow \text{Expand}(\text{seed}_{\text{PKE}} \parallel \text{"h"}, n)$. This is the same deterministic expansion used during key generation, so the encryptor obtains the identical h without requiring it to be transmitted explicitly.

Steps 2–4: Asymmetric ephemeral vector sampling. Three sparse ephemeral vectors are sampled deterministically from the randomness seed θ :

$$r_1 \xleftarrow{\$} R_{w_{r_1}}, \quad \text{with } \text{wt}(r_1) = w_{r_1}, \quad (33)$$

$$r_2 \xleftarrow{\$} R_{w_{r_2}}, \quad \text{with } \text{wt}(r_2) = w_{r_2}, \quad (34)$$

$$e \xleftarrow{\$} R_{w_e}, \quad \text{with } \text{wt}(e) = w_e. \quad (35)$$

This is the first key innovation of NSS-HQC. In the standard HQC specification, all three vectors share the same weight $w_r = w_e$. In NSS-HQC, the weights are *asymmetric*: $w_{r_1} = w_{r_2} < w_r^{\text{std}}$ and $w_e > w_e^{\text{std}}$, while maintaining the total weight budget $w_{r_1} + w_{r_2} + w_e = 2w_r^{\text{std}} + w_e^{\text{std}}$.

The rationale for this rebalancing is as follows. On the codeword support, the structural composite decryption error is $e'_{\text{struct}} = \pi_{n_C}(x \cdot r_2 + r_1 \cdot y + e)$. The terms $x \cdot r_2$ and $r_1 \cdot y$ involve multiplication by the secret vectors (x, y) of weight w_{sk} , which *amplifies* the error: each nonzero coordinate of r_2 (resp. r_1) contributes approximately w_{sk} error bits through the convolution. In contrast, e contributes directly without amplification. Therefore, reducing w_{r_1} and w_{r_2} (the amplified terms) while increasing w_e (the non-amplified term) decreases the total expected error weight, leading to a lower DFR and hence allowing shorter code parameters.

Each vector is sampled using domain-separated expansion from θ : the tags "r1", "r2", "e" ensure that the three vectors are computationally independent even though they share the same seed. The sampling uses constant-time rejection sampling to prevent side-channel leakage of the support positions.

Step 5: First ciphertext component. The first ciphertext component is computed as

$$u = r_1 + h \cdot r_2 \in R. \quad (36)$$

This has the same algebraic structure as the public syndrome $s = x + h \cdot y$: it is a noisy linear combination of the public vector h with sparse coefficients. Its marginal distribution is the generalized DQCSD(n, w_{r_1}, w_{r_2}) distribution. In the IND-CPA proof, however, u is analyzed together with the second-component mask because both contain the same ephemeral vector r_2 . The ring multiplication $h \cdot r_2$ is the dominant computational cost of encryption.

Step 6: Second ciphertext component (before compression). The uncompressed second ciphertext component is

$$v_{\text{raw}} = C.\text{Encode}(m) + \pi_{n_C}(s \cdot r_2 + e) \in \mathbb{F}_2^{n_C}. \quad (37)$$

This equation has three additive terms, each serving a distinct purpose:

- $C.\text{Encode}(m) \in \mathbb{F}_2^{n_C}$: the error-corrected encoding of the plaintext message m . This is the “payload” that the receiver must recover.
- $\pi_{n_C}(s \cdot r_2) \in \mathbb{F}_2^{n_C}$: the masking term restricted to the codeword support. Since $s = x + h \cdot y$ and (h, s) are public, the product $s \cdot r_2$ effectively creates a one-time-pad-like mask over the encoded message. Without knowledge of r_2 (which is ephemeral and secret), this mask is covered by the joint PKE-core indistinguishability assumption used in the IND-CPA proof.
- $\pi_{n_C}(e) \in \mathbb{F}_2^{n_C}$: the additive noise on the codeword support. This term ensures that even if r_2 were somehow recovered, the noise e provides an additional layer of protection. Together with $\pi_{n_C}(x \cdot r_2 + r_1 \cdot y)$, which appears after decryption, this forms the structural part of the composite error that the decoder must correct.

The projection π_{n_C} is essential: the ring products are length- n objects, while $C.\text{Encode}(m)$ is a length- n_C codeword. Only the first n_C ring coordinates are mixed into the payload component.

Step 7: Dither generation. The pseudorandom dither is generated at block level as described in Section 2.7.3. A base dither $d_{\text{base}} \in \mathbb{F}_2^{n_1 \cdot n_2}$ (one bit per RM code bit) is generated from salt and pk, then expanded to $d \in \mathbb{F}_2^{n_C}$ by replicating each bit mult times:

$$d_{\text{base}} = \text{Expand}(\text{salt} \parallel \text{pk} \parallel \text{"dither"}, n_1 \cdot n_2), \quad d = \text{dup}_{\text{mult}}(d_{\text{base}}). \quad (38)$$

The block-level structure of d ensures that after decompression and dither removal during decryption, the RM duplication alignment is preserved (see Remark 6).

Step 8: Ciphertext compression. **This is the second key innovation of NSS-HQC.** The dithered ciphertext is compressed blockwise:

$$\hat{v} = Q.\text{CompressBlockwise}(v_{\text{raw}} \oplus d) \in \{0, \dots, Q-1\}^{n_1 \cdot n_2}. \quad (39)$$

Each group of mult consecutive bits is mapped to a single b -bit symbol, reducing the representation of the v component from $n_C = n_1 \cdot n_2 \cdot \text{mult}$ bits to $n_1 \cdot n_2 \cdot b$ bits. The compression ratio on v is $b/\text{mult} = 2/5 = 40\%$ for the uniform choice $(\text{mult}, Q) = (5, 4)$ across all four security levels.

Step 9: Ciphertext output. The ciphertext $\text{ct} = (u, \hat{v})$ consists of the uncompressed first component $u \in R$ (encoded as $\lceil n/8 \rceil$ bytes) and the compressed second component $\hat{v}$ (encoded as $\lceil n_1 \cdot n_2 \cdot b/8 \rceil$ bytes).

2.8.3 Decrypt Algorithm

Algorithm 3. NSS-HQC-PKE.Decrypt(sk, ct, salt)

Require: Secret key sk, ciphertext $\text{ct} = (u, \hat{v})$, salt

Ensure: Message m' or $\perp$

```

1: Parse sk as (seedsk,  $\sigma$ , pk)
2: (seedx, seedy)  $\leftarrow I(\text{seed}_{\text{sk}})$ 
3:  $\triangleright$  only y is needed for PKE decryption
4:  $y \leftarrow R_{w_{\text{sk}}}$  using Expand(seedy,  $\cdot$ )
5:  $d_{\text{base}} \leftarrow \text{Expand}(\text{salt} \parallel \text{pk} \parallel \text{"dither"}, n_1 \cdot n_2)$ ;  $d \leftarrow \text{dup}_{\text{mult}}(d_{\text{base}})$ 
6:  $\triangleright$  block-level dither
7:  $\tilde{v} \leftarrow Q.\text{DecompressBlockwise}(\hat{v}) \oplus d$ 
8:  $w \leftarrow \tilde{v} - \pi_{n_C}(u \cdot y)$ 
9: for  $i = 1$  to  $n_1$  do
10:    $(s_i, \rho_i) \leftarrow \text{RM.Decode}(w[\text{block}_i])$ 
11: end for
12: candidates  $\leftarrow \{i : \rho_i < \tau\}$ 
13: erasure_mask  $\leftarrow$  bottom- $\epsilon_{\text{max}}$  of candidates ranked by  $\rho_i$ 
14:  $\triangleright$  cap at  $\epsilon_{\text{max}} = \lfloor (\delta_1 - 1)/2 \rfloor$ 
15:  $m' \leftarrow \text{RS.DecodeErrorsAndErasures}((s_1, \dots, s_{n_1}), \text{erasure\_mask})$ 
16: if  $m' = \text{FAIL}$  then return  $\perp$ 
17: end if
18: return  $m'$ 

```

Detailed explanation of each step:

Step 1: Secret key parsing. The secret key is parsed into its three components: the secret seed seed_{sk} , the rejection secret σ , and the public key pk.

Steps 2–4: Secret vector regeneration. The secret seed seed_{sk} is split into sub-seeds $(\text{seed}_x, \text{seed}_y) \leftarrow I(\text{seed}_{\text{sk}})$ via SHA3-512, but PKE decryption only expands y . The vector x is not needed by the decryption algorithm itself; it appears only in the algebraic correctness proof

because $s = x + h \cdot y$. This avoids storing the full sparse vectors in the secret key and saves one fixed-weight expansion during decryption compared with regenerating both (x, y) .

Step 5: Dither reconstruction. The base dither d_{base} is recomputed from the salt and pk, then expanded to the full dither $d = \text{dup}_{\text{mult}}(d_{\text{base}})$. Since the same deterministic expansion is used as in encryption, d is identical to the dither used during compression. This step can be performed *before* any decryption attempt, avoiding the circular dependency that would arise if d depended on θ or m .

Step 6: Ciphertext decompression. The compressed ciphertext $\hat{v}$ is decompressed and un-dithered:

$$\tilde{v} = Q.\text{DecompressBlockwise}(\hat{v}) \oplus d \in \mathbb{F}_2^{nC}. \quad (40)$$

Because the decompression outputs constant blocks and the dither within each group is also constant, the result $\tilde{v}$ preserves the RM duplication structure: within each group of mult bits, all bits are equal. The decompressed vector $\tilde{v}$ is an approximation of the original v_{raw} , differing by the quantization error $\hat{e} = \tilde{v} - v_{\text{raw}}$.

Step 7: Syndrome extraction (noise cancellation). The core decryption operation computes:

$$w = \tilde{v} - \pi_{nC}(u \cdot y) \in \mathbb{F}_2^{nC}. \quad (41)$$

Since all arithmetic is over $\mathbb{F}_2$, subtraction equals addition. We now show that w equals the encoded message plus a composite error. Substituting the encryption equations:

$$\begin{aligned} w &= \tilde{v} - \pi_{nC}(u \cdot y) \\ &= (v_{\text{raw}} + \hat{e}) - \pi_{nC}((r_1 + h \cdot r_2) \cdot y) \\ &= C.\text{Encode}(m) + \pi_{nC}(s \cdot r_2 + e) + \hat{e} - \pi_{nC}(r_1 \cdot y + h \cdot r_2 \cdot y) \\ &= C.\text{Encode}(m) + \pi_{nC}((x + h \cdot y) \cdot r_2 + e + r_1 \cdot y + h \cdot r_2 \cdot y) + \hat{e} \\ &= C.\text{Encode}(m) + \pi_{nC}(x \cdot r_2 + \cancel{h \cdot y \cdot r_2} + e + r_1 \cdot y + \cancel{h \cdot r_2 \cdot y}) + \hat{e} \\ &= C.\text{Encode}(m) + \underbrace{\pi_{nC}(x \cdot r_2 + r_1 \cdot y + e)}_{e'_{\text{struct}}} + \underbrace{\hat{e}}_{e_{\text{quant}}}. \end{aligned} \quad (42)$$

The terms $h \cdot y \cdot r_2$ and $h \cdot r_2 \cdot y$ cancel because ring multiplication is commutative and all additions are over $\mathbb{F}_2$. The result is $w = C.\text{Encode}(m) + e''$, where $e'' = e'_{\text{struct}} + \hat{e}$ is the total composite error on the codeword support. The structural term comes from the HQC algebra, while $\hat{e}$ comes from the deterministic quantization layer.

Steps 8–9: Inner decoding with reliability extraction. The noisy codeword $w \in \mathbb{F}_2^{nC}$ is partitioned into n_1 blocks, each of length $128 \cdot \text{mult}$. The i -th block is decoded by the duplicated RM decoder via FHT:

$$(s_i, \rho_i) \leftarrow \text{RM.Decode}(w[\text{block}_i]), \quad i = 1, \dots, n_1, \quad (43)$$

producing a decoded RS symbol $s_i \in \mathbb{F}_{256}$ and a reliability measure $\rho_i \in [0, 1]$. If the composite error weight in block i is below the correction threshold $64 \cdot \text{mult}/2$, then s_i is correctly decoded and ρ_i is high. Otherwise, s_i may be incorrect and ρ_i is low.

Steps 10–11: Erasure marking with cap. Symbols with reliability below the threshold τ are identified as erasure candidates. To prevent the number of erasures from exceeding the RS decoder's capacity, the erasure count is capped at $\epsilon_{\text{max}} = \lfloor (\delta_1 - 1)/2 \rfloor$. If more candidates exist than the cap allows, only the ϵ_{max} least reliable symbols are declared as erasures. This

guarantees that the RS decoder always has at least $\lfloor (\delta_1 - 1)/2 \rfloor$ units of redundancy available for error correction, regardless of the noise realization.

Step 12: Outer decoding (errors-and-erasures). The RS decoder receives n_1 symbols $(s_1, \dots, s_{n_1})$ together with the capped erasure mask, and performs errors-and-erasures decoding using the Berlekamp–Massey algorithm with Forney’s formula (Section 2.6.3):

$$m' \leftarrow \text{RS.DecodeErrorsAndErasures}((s_1, \dots, s_{n_1}), \text{erasure_mask}). \quad (44)$$

Because $|\text{erasure_mask}| \leq \epsilon_{\max} = \lfloor (\delta_1 - 1)/2 \rfloor$, the decoder can still correct up to $\lfloor (\delta_1 - 1 - \epsilon)/2 \rfloor$ additional errors beyond the declared erasures.

Steps 13–15: Output. If decoding fails, $\perp$ is returned. Otherwise, the recovered message m' is returned. In the KEM context (Algorithm 6), a decoding failure triggers implicit rejection rather than exposing $\perp$ to the caller.

2.9 NSS-HQC-KEM

The KEM is obtained from NSS-HQC-PKE by applying the Fujisaki–Okamoto (FO) transform [12] with implicit rejection. The FO transform converts any IND-CPA-secure PKE into an IND-CCA2-secure KEM by deriving the encryption randomness from the message via a hash function, and verifying correctness by re-encrypting during decapsulation.

2.9.1 KEM Key Generation

Algorithm 4. NSS-HQC-KEM.KeyGen

- 1: $(\text{pk}, \text{sk}) \leftarrow \text{NSS-HQC-PKE.KeyGen}()$
 - 2: $\text{ek} \leftarrow \text{pk}; \text{dk} \leftarrow \text{sk}$
 - 3: **return** (ek, dk)
-

The KEM key generation is a thin wrapper around the PKE key generation. The encapsulation key ek is simply the PKE public key, and the decapsulation key dk is the PKE secret key (which includes σ for implicit rejection).

2.9.2 Encapsulation Algorithm

Algorithm 5. NSS-HQC-KEM.Encaps(ek)

Require: Encapsulation key ek

Ensure: Shared secret K , ciphertext ct_{full}

- 1: $m \xleftarrow{\$} \{0, 1\}^{8k_1}; \text{salt} \xleftarrow{\$} \{0, 1\}^{\lambda_{\text{salt}}}$
 - 2: $\theta \leftarrow G(m \parallel \text{salt} \parallel \text{ek})$
 - 3: $\text{ct} \leftarrow \text{NSS-HQC-PKE.Encrypt}(\text{ek}, m, \theta, \text{salt})$
 - 4: $\text{ct}_{\text{full}} \leftarrow \text{ct} \parallel \text{salt}$
 - 5: $K \leftarrow H_K(m \parallel \text{ct}_{\text{full}})$
 - 6: **return** $(K, \text{ct}_{\text{full}})$
-

Detailed explanation:

Step 1: Plaintext and salt sampling. A uniformly random plaintext $m \in \{0, 1\}^{8k_1}$ and a uniformly random salt $\text{salt} \in \{0, 1\}^{\lambda_{\text{salt}}}$ are sampled. The plaintext m is *not* the actual shared secret; rather, it is an intermediate value from which both the encryption randomness and the shared secret are derived. The plaintext length $8k_1$ matches the message capacity of the concatenated code C (which encodes k_1 bytes = $8k_1$ bits), while the shared secret length κ may differ (e.g., for NSS-HQC-128, $8k_1 = 128$ bits but $\kappa = 256$ bits). The salt provides domain separation across multiple encapsulations under the same key, preventing multi-target attacks on the hash function.

Step 2: Derandomized encryption. The encryption randomness θ is derived deterministically from the plaintext, salt, and encapsulation key via the hash function G :

$$\theta = G(m \parallel \text{salt} \parallel \text{ek}). \quad (45)$$

This derandomization is the core mechanism of the FO transform: it ties the encryption randomness to the plaintext, enabling the decapsulator to verify the ciphertext by re-encrypting with the recovered plaintext and checking for consistency.

Step 3: PKE encryption. The PKE encryption algorithm is invoked with the derandomized randomness θ and the salt. Since θ is derived from m , the encryption is a deterministic function of $(m, \text{salt}, \text{ek})$.

Step 4: Shared secret derivation. The shared secret K is derived by hashing the plaintext together with the full ciphertext and salt:

$$K = H_{\kappa}(m \parallel \text{ct}_{\text{full}}) \in \{0, 1\}^{\kappa}. \quad (46)$$

Including $\text{ct}_{\text{full}} = \text{ct} \parallel \text{salt}$ in the hash input binds the shared secret to the specific ciphertext and its salt, ensuring that different ciphertexts (even those encrypting the same m) produce different shared secrets.

Steps 5–6: Output. The full ciphertext $\text{ct}_{\text{full}} = \text{ct} \parallel \text{salt}$ includes the salt as a suffix. The shared secret K and the full ciphertext are returned.

2.9.3 Decapsulation Algorithm

Algorithm 6. NSS-HQC-KEM.Decaps($\text{dk}, \text{ct}_{\text{full}}$)

Require: Decapsulation key dk , ciphertext ct_{full}

Ensure: Shared secret K'

- 1: Parse dk as $(\text{seed}_{\text{sk}}, \sigma, \text{pk})$; parse ct_{full} as (ct, salt)
 - 2: $m' \leftarrow \text{NSS-HQC-PKE.Decrypt}(\text{dk}, \text{ct}, \text{salt})$
 - 3: $\bar{m} \leftarrow \text{Select}(m' = \perp, 0^{8k_1}, m')$
 - 4: $\theta' \leftarrow G(\bar{m} \parallel \text{salt} \parallel \text{pk})$
 - 5: $\text{ct}' \leftarrow \text{NSS-HQC-PKE.Encrypt}(\text{pk}, \bar{m}, \theta', \text{salt})$
 - 6: $b_{\text{ok}} \leftarrow (m' \neq \perp) \wedge (\text{ct}' = \text{ct})$
 - 7: $K_{\text{acc}} \leftarrow H_{\kappa}(\bar{m} \parallel \text{ct}_{\text{full}})$
 - 8: $K_{\text{rej}} \leftarrow H_{\kappa}(\sigma \parallel \text{ct}_{\text{full}})$
 - 9: $K' \leftarrow \text{CTSelect}(b_{\text{ok}}, K_{\text{acc}}, K_{\text{rej}})$
 - 10: **return** K'
-

Detailed explanation:

Step 1: Parsing. The decapsulation key and ciphertext are parsed into their constituent parts. The salt is extracted from the ciphertext suffix.

Step 2: PKE decryption. The PKE decryption algorithm (Algorithm 3) is invoked to recover the candidate plaintext m' . Note that salt is passed as a separate argument so that the dither vector d can be reconstructed for ciphertext decompression. If the composite error exceeds the decoding capacity of C , decryption returns $\perp$.

Step 3: Failure masking. If PKE decryption fails ($m' = \perp$), the algorithm replaces the invalid value by a fixed dummy message $\bar{m} = 0^{8k_1}$ for the purpose of the re-encryption check. If decryption succeeds, $\bar{m} = m'$. This step is a constant-time selection and does not reveal whether decoding succeeded.

Step 4: Re-derive encryption randomness. The encryption randomness θ' is re-derived from $\bar{m}$ using the same deterministic hash:

$$\theta' = G(\bar{m} \parallel \text{salt} \parallel \text{pk}). \quad (47)$$

If decryption was correct (i.e., $m' = m$), then $\theta' = \theta$, and the re-encryption in the next step will reproduce the original ciphertext.

Step 5: Re-encryption verification. The ciphertext is re-computed from $\bar{m}$:

$$\text{ct}' = \text{NSS-HQC-PKE.Encrypt}(\text{pk}, \bar{m}, \theta', \text{salt}). \quad (48)$$

This is the ‘‘Fujisaki–Okamoto check’’. The acceptance bit is

$$b_{\text{ok}} = (m' \neq \perp) \wedge (\text{ct}' = \text{ct}).$$

Thus a decoding failure and a re-encryption mismatch are handled by the same implicit-rejection path.

Steps 6–9: Constant-time conditional output. The decapsulator computes both candidate outputs:

$$K_{\text{acc}} = H_{\kappa}(\bar{m} \parallel \text{ct}_{\text{full}}), \quad K_{\text{rej}} = H_{\kappa}(\sigma \parallel \text{ct}_{\text{full}}),$$

and returns K_{acc} if $b_{\text{ok}} = 1$, otherwise K_{rej} . Since σ is unknown to any party other than the key holder, K_{rej} is indistinguishable from a random value to an attacker. This prevents chosen-ciphertext attackers from using decoding failures or re-encryption mismatches as an oracle.

Constant-time execution. The comparison, acceptance-bit computation, and final selection are implemented by constant-time operations. The algorithmic listing therefore matches the side-channel requirement used in the security discussion.

2.10 Correctness

Theorem 7 (Correctness of NSS-HQC). *Let ek and dk be generated by $\text{NSS-HQC-KEM.KeyGen}$. Let $(K, \text{ct}_{\text{full}}) \leftarrow \text{Encaps}(\text{ek})$. Then*

$$\Pr[\text{Decaps}(\text{dk}, \text{ct}_{\text{full}}) \neq K] \leq \text{DFR}_{\text{NSS}}.$$

Proof. We trace the execution of Encaps and Decaps step by step, showing that the shared secrets match whenever decryption succeeds.

Encapsulation side. The encapsulator produces:

$$u = r_1 + h \cdot r_2, \quad (49)$$

$$v_{\text{raw}} = C.\text{Encode}(m) + \pi_{n_C}(s \cdot r_2 + e), \quad (50)$$

$$d_{\text{base}} = \text{Expand}(\text{salt} \parallel \text{pk} \parallel \text{"dither"}, n_1 \cdot n_2), d = \text{dup}_{\text{mult}}(d_{\text{base}}), \quad (51)$$

$$\hat{v} = Q.\text{CompressBlockwise}(v_{\text{raw}} \oplus d). \quad (52)$$

The ciphertext is $\text{ct} = (u, \hat{v})$, the transmitted ciphertext is $\text{ct}_{\text{full}} = \text{ct} \parallel \text{salt}$, and the shared secret is $K = H_\kappa(m \parallel \text{ct}_{\text{full}})$.

Decapsulation side. The decapsulator receives (ct, salt) and computes:

Step (a): Dither reconstruction. Since salt and pk are known, the decapsulator computes the identical dither vector:

$$d'_{\text{base}} = \text{Expand}(\text{salt} \parallel \text{pk} \parallel \text{"dither"}, n_1 \cdot n_2), d' = \text{dup}_{\text{mult}}(d'_{\text{base}}) = d. \quad (53)$$

Step (b): Decompression. The decompressed ciphertext is:

$$\tilde{v} = Q.\text{DecompressBlockwise}(\hat{v}) \oplus d. \quad (54)$$

Define the quantization error as $\hat{e} = \tilde{v} - v_{\text{raw}}$. This is the exact deterministic discrepancy induced by the dithered compress-decompress cycle for the given ciphertext; the DFR analysis later upper-bounds its effect by a conservative proxy channel.

Step (c): Noise cancellation. The decapsulator computes $w = \tilde{v} - \pi_{n_C}(u \cdot y)$. We expand this using the definitions above. First, substitute $\tilde{v} = v_{\text{raw}} + \hat{e}$:

$$w = v_{\text{raw}} + \hat{e} - \pi_{n_C}(u \cdot y). \quad (55)$$

Next, substitute $v_{\text{raw}} = C.\text{Encode}(m) + \pi_{n_C}(s \cdot r_2 + e)$ and $u = r_1 + h \cdot r_2$:

$$w = C.\text{Encode}(m) + \pi_{n_C}(s \cdot r_2 + e) + \hat{e} - \pi_{n_C}((r_1 + h \cdot r_2) \cdot y). \quad (56)$$

Expand $s = x + h \cdot y$ and distribute the multiplication:

$$w = C.\text{Encode}(m) + \pi_{n_C}((x + h \cdot y) \cdot r_2 + e + r_1 \cdot y + h \cdot r_2 \cdot y) + \hat{e}. \quad (57)$$

Since R is commutative, $h \cdot y \cdot r_2 = h \cdot r_2 \cdot y$. These terms cancel:

$$w = C.\text{Encode}(m) + \pi_{n_C}(x \cdot r_2 + r_1 \cdot y + e) + \hat{e}. \quad (58)$$

Therefore:

$$\boxed{w = C.\text{Encode}(m) + e'', \quad e'' = \pi_{n_C}(x \cdot r_2 + r_1 \cdot y + e) + \hat{e}.} \quad (59)$$

Step (d): Decoding. The concatenated decoder $C.\text{Decode}$ is applied to w . If the Hamming weight of e'' (restricted to the codeword support) is within the correction capability of C , the decoder successfully recovers $m' = m$. The probability of failure is DFR_{NSS} , analyzed in Section 3.

Step (e): Re-encryption verification. When $m' = m$, the re-derived randomness is $\theta' = G(m \parallel \text{salt} \parallel \text{pk}) = \theta$, and re-encryption produces $\text{ct}' = \text{ct}$ by the determinism of Algorithm 2. Therefore the FO check passes and $K' = H_\kappa(m \parallel \text{ct}_{\text{full}}) = K$.

Conclusion. The decapsulation fails (i.e., $K' \neq K$) only when the decoder fails to recover m from w , which occurs with probability at most DFR_{NSS} . $\square$

2.11 Parameter Sets

Table 1. NSS-HQC parameter sets for four security levels.

Parameter set	Cl.	Qu.	n	w_{sk}	w_{r_1}	w_{r_2}	w_e	n_1	k_1	mult	Q	b
NSS-HQC-128	128	80	29,443	73	66	66	117	46	16	5	4	2
NSS-HQC-256	256	128	54,493	117	106	106	187	70	32	5	4	2
NSS-HQC-384	384	192	122,579	175	159	159	279	100	48	5	4	2
NSS-HQC-512	512	256	217,901	233	213	213	372	130	64	5	4	2

Table 2. Derived sizes (bytes). The “ct baseline” column is NSS-HQC without quantization (M2 disabled) at the same $(n, n_1, k_1, \text{mult})$; “ct reduction” is the relative saving from enabling quantization only.

Scheme	pk	ct (baseline, no quant.)	ct (NSS)	pk+ct (NSS)	ct reduction
NSS-HQC-128	3,713	7,361	5,153	8,866	−30.0%
NSS-HQC-256	6,844	12,412	9,052	15,896	−27.1%
NSS-HQC-384	15,371	23,323	18,523	33,894	−20.6%
NSS-HQC-512	27,302	37,638	31,398	58,700	−16.6%

Note: “ct (NSS)” is the PKE ciphertext body $|u| + |\hat{v}|$, excluding the $\lambda_{\text{salt}}/8$ -byte salt suffix appended during KEM encapsulation. KEM-level ciphertext totals (PKE body + salt) are reported in Table 15.

Table 3. Additional parameter-derived quantities.

Parameter set	κ	λ_{salt}	λ_{seed}	n_C	ct_v bits (NSS)
NSS-HQC-128	256	256	256	29,440	11,776
NSS-HQC-256	256	256	256	44,800	17,920
NSS-HQC-384	384	384	384	64,000	25,600
NSS-HQC-512	512	512	512	83,200	33,280

The encapsulated key length κ is set to $\max(256, \text{classical_target})$, satisfying the requirement that κ is not less than the classical security strength.

Table 4. Erasure threshold τ and RS decoding parameters for each security level.

Parameter set	τ	$\delta_1 = n_1 - k_1 + 1$	Max errors (hard)	ϵ_{max}	ρ metric
NSS-HQC-128	0.30	31	15	15	Eq. (13)
NSS-HQC-256	0.28	39	19	19	Eq. (13)
NSS-HQC-384	0.26	53	26	26	Eq. (13)
NSS-HQC-512	0.25	67	33	33	Eq. (13)

Interpretation of τ values. The threshold τ is a dimensionless real number in $[0, 1]$, with the same scale as the reliability metric ρ_i defined in Eq. (13). A block is marked as an erasure candidate if $\rho_i < \tau$.

Erasure cap. To guarantee that the RS decoder always has sufficient capacity, the number of erasures is capped at $\epsilon_{\max} = \lfloor (\delta_1 - 1)/2 \rfloor$. If more than $\epsilon_{\max}$ symbols satisfy $\rho_i < \tau$, only the $\epsilon_{\max}$ symbols with the lowest reliability are declared as erasures. The remaining symbols are treated as (potentially incorrect) hard decisions. Formally:

$$\text{erasure_mask} = \text{bottom-}\epsilon_{\max} \text{ of } \{i : \rho_i < \tau\} \text{ ranked by } \rho_i. \quad (60)$$

This ensures that $|\text{erasure_mask}| \leq \epsilon_{\max} = \lfloor (\delta_1 - 1)/2 \rfloor$, which always satisfies the RS capacity constraint $2t' + \epsilon \leq \delta_1 - 1$ as long as the number of uncaught errors t' satisfies $t' \leq \lfloor (\delta_1 - 1)/2 \rfloor$.

Threshold selection. The values of τ are determined by DFR-optimal grid search and are chosen conservatively: they mark only the lowest-reliability symbols as erasures and rely on the erasure cap to prevent overflow. The values listed above are the operating points used by the final parameter table.

Remark 8 (Parameter status). *The parameters in Tables 1–4 are the selected values used by NSS-HQC. The reproducibility material records the following checks for each row:*

1. *Each n is a primitive prime with $\text{ord}_n(2) = n - 1$.*
2. *The ISD costs are evaluated with the same calibrated estimator methodology as the security analysis.*
3. *The DFR values are evaluated with the exact RM weight enumerator used in the conservative upper bound.*
4. *The τ values come from the DFR-optimal grid search for the final (w_{r_1}, w_{r_2}, w_e) triple.*

There are no hidden parameters beyond those listed in Tables 1–4. The internal byte ordering follows little-endian convention throughout, consistent with the standard HQC specification [11]. The inner code block partitioning is sequential: the i -th RS symbol corresponds to coordinates $[(i - 1) \cdot 128 \cdot \text{mult}, i \cdot 128 \cdot \text{mult})$ of the codeword vector.

3 Unified Decryption Failure Rate Analysis

This section develops a unified analytical framework for bounding the decryption failure rate (DFR) of NSS-HQC. The DFR is the probability that the concatenated decoder $C.\text{Decode}$ fails to recover the message m from the noisy codeword $w = C.\text{Encode}(m) + e''$. The composite error e'' contains the projected HQC structural error and the quantization discrepancy introduced by ciphertext compression, and the unified framework accounts for both simultaneously.

3.1 Composite Error Model

From Eq. (59), the total error seen by the decoder is:

$$e'' = \underbrace{\pi_{n_C}(x \cdot r_2 + r_1 \cdot y + e)}_{e'_{\text{struct}}} + \underbrace{\hat{e}}_{\hat{e}_{\text{quant}}}, \quad (61)$$

where:

- e'_{struct} is the **structural error** on the first n_C coordinates, arising from the HQC encryption mechanism. It consists of three projected terms: $x \cdot r_2$ (secret-ephemeral cross product), $r_1 \cdot y$ (ephemeral-secret cross product), and e (additive noise).
- $\hat{e}_{\text{quant}}$ is the **quantization error**, arising from the compress-decompress cycle applied to the ciphertext component v .

The random sources used to generate the dither are independent of the secret key and the ephemeral vectors. However, $\hat{e}_{\text{quant}}$ is a deterministic function of the dithered v_{raw} , and therefore the proof does not rely on a literal coordinate-wise independence claim for $\hat{e}_{\text{quant}}$. Instead, the closed-form DFR calculation below uses an explicit conservative proxy channel for the quantization layer.

3.2 Structural Error Distribution

We model each coordinate of e'_{struct} as an independent Bernoulli random variable with crossover probability p_{struct} . This is the **PEB-BSC (Positional Error Bound – Binary Symmetric Channel)** model used in [4].

For a specific codeword coordinate position $j \in \{0, 1, \dots, n_C - 1\}$, the j -th bit of e'_{struct} is:

$$e'_{\text{struct},j} = (x \cdot r_2)_j \oplus (r_1 \cdot y)_j \oplus e_j. \quad (62)$$

The probability that $e'_{\text{struct},j} = 1$ is:

$$p_{\text{struct}} = \Pr[e'_{\text{struct},j} = 1] = \frac{1}{2} (1 - (1 - 2p_{xr_2})(1 - 2p_{r_1y})(1 - 2p_e)), \quad (63)$$

where:

- $p_{xr_2} = \Pr[(x \cdot r_2)_j = 1]$ is the probability that the j -th coordinate of the convolution $x \cdot r_2$ is 1. For random positions of the nonzero entries in x and r_2 , this is bounded by $p_{xr_2} \leq \frac{1}{2} (1 - (1 - 2/n)^{w_{\text{sk}} \cdot w_{r_2}}) \approx w_{\text{sk}} \cdot w_{r_2} / n$ for small $w_{\text{sk}} \cdot w_{r_2} / n$.
- p_{r_1y} is defined analogously with w_{r_1} and w_{sk} .
- $p_e = w_e / n$ is the probability that coordinate j of the additive noise e is nonzero.

Substituting the exact per-coordinate bounds from the bullet items above, the conservative upper bound is:

$$p_{\text{struct}} \leq \frac{1}{2} \left(1 - \left(1 - \frac{2}{n} \right)^{w_{\text{sk}} \cdot w_{r_1}} \left(1 - \frac{2}{n} \right)^{w_{\text{sk}} \cdot w_{r_2}} \left(1 - \frac{2}{n} \right)^{w_e} \right). \quad (64)$$

Purpose of asymmetric rebalancing. The expression $(1 - 2w_{\text{sk}} \cdot w/n)$ decreases exponentially in w when w is multiplied by w_{sk} (the amplification factor), but only linearly when w appears alone (as in the w_e term). Therefore, transferring weight from the amplified terms (w_{r_1}, w_{r_2}) to the non-amplified term (w_e) increases the overall product, which *decreases* p_{struct} . This is the mathematical basis for the asymmetric weight optimization.

3.3 Quantization Error Distribution

Under block-level dithering (Section 2.7.3) with constant-block decompression, the decompressed value $\tilde{v}$ is constant within each RM duplication group before the subtraction of $\pi_{n_C}(u \cdot y)$. The actual quantization error $\hat{e} = \tilde{v} - v_{\text{raw}}$ need not itself be constant within the group, because v_{raw} already includes the structural HQC mask and noise. Therefore, the DFR analysis separates the implemented algorithm from the analysis abstraction: the implementation uses deterministic hard decompression, while the analysis upper-bounds its effect by a conservative hard-decision proxy on the underlying RM coordinate.

For DFR estimation we use a conservative hard-decision proxy for one duplication group. If each bit in a group is independently flipped with probability p_{bit} under the quantization-channel estimator for the chosen (Q, mult) , the probability that this proxy flips the underlying

RM coordinate is:

$$p_{\text{quant}} = \sum_{k=\lceil \text{mult}/2 \rceil}^{\text{mult}} \binom{\text{mult}}{k} p_{\text{bit}}^k (1 - p_{\text{bit}})^{\text{mult}-k}. \quad (65)$$

For example, if the estimator gives $p_{\text{bit}} = 0.30$ at $(Q, \text{mult}) = (4, 5)$, then $p_{\text{quant}} = 10 \times 0.30^3 \times 0.70^2 + 5 \times 0.30^4 \times 0.70 + 0.30^5 \approx 0.163$. The concrete value of p_{bit} is an output of the quantization-channel estimator used in parameter search, not a free fitting parameter. This quantity is used as an analysis proxy for the duplication gain; it is not a statement that the implemented RM decoder first hard-votes the duplicated bits.

3.4 Combined Error Probability

With block-level dithering and constant-block decompression, the implemented RM decoder keeps the mult received copies as evidence and runs the FHT decoder described in Section 2.6.1. For the closed-form DFR bound, we upper-bound the two-layer behavior by a single effective per-RM-coordinate crossover probability. In the conservative proxy, the structural layer contributes p_{struct} and the quantization layer contributes p_{quant} ; since the two proxy layers are combined by XOR over $\mathbb{F}_2$, the effective rate is the BSC cascade:

$$p_{\text{eff}} = p_{\text{struct}}(1 - p_{\text{quant}}) + (1 - p_{\text{struct}})p_{\text{quant}} = p_{\text{struct}} + p_{\text{quant}} - 2p_{\text{struct}}p_{\text{quant}}. \quad (66)$$

The key insight of NSS-HQC. Reducing p_{struct} through asymmetric rebalancing also reduces the impact of the quantization layer in the cascade term $-2p_{\text{struct}}p_{\text{quant}}$. Thus the two optimizations are not analyzed as independent bandwidth tricks; they are coupled through the same effective error probability that drives the RM and RS failure probabilities. The actual decoder may perform better than this proxy because it processes the full duplicated RM block rather than first collapsing it to one hard bit.

3.5 DFR Upper Bound

Theorem 9 (Unified DFR bound). *Let $P_{\text{RM}}(p)$ denote the ML decoding failure probability of $\text{RM}(1, 7) = [128, 8, 64]$ under $\text{BSC}(p)$. Under the block-level quantization proxy with effective per-RM-bit error rate p_{eff} (Eq. 66), define for one inner block:*

$$P_{\text{err}} = \Pr[\text{RM output is wrong and not erased}], \quad P_{\text{era}} = \Pr[\text{block is declared as an erasure}],$$

where the erasure event is determined by the reliability threshold τ and the cap in Eq. (60). The errors-and-erasures RS decoder fails only if the number t of unerased wrong symbols and the number ϵ of erasures violate $2t + \epsilon \leq \delta_1 - 1$. Therefore the DFR is bounded by:

$$\text{DFR}_{\text{NSS}} \leq \sum_{\substack{t, \epsilon \geq 0 \\ t + \epsilon \leq n_1 \\ 2t + \epsilon > \delta_1 - 1}} \frac{n_1!}{t! \epsilon! (n_1 - t - \epsilon)!} P_{\text{err}}^t P_{\text{era}}^\epsilon (1 - P_{\text{err}} - P_{\text{era}})^{n_1 - t - \epsilon}. \quad (67)$$

As a conservative fallback, one may set $P_{\text{era}} = 0$ and $P_{\text{err}} = P_{\text{RM}}(p_{\text{eff}})$, which recovers the hard-decision RS bound. The submitted parameter search uses the reliability threshold τ to estimate $(P_{\text{err}}, P_{\text{era}})$ under the same proxy channel, so the erasure benefit is accounted for through the RS condition above rather than by an informal tightening factor.

The ML failure probability $P_{\text{RM}}(p)$ for RM(1,7) under BSC(p) is bounded by:

$$P_{\text{RM}}(p) \leq \sum_{w=32}^{128} A_w \cdot p^w \cdot (1-p)^{128-w}, \quad (68)$$

where A_w is the weight enumerator of RM(1,7) and $32 = d/2$ is the ML decoding radius. The implemented decoder operates on the full duplicated block through soft-combined FHT. The bound above evaluates the base RM(1,7) failure probability at the conservative proxy rate p_{eff} from Eq. (66), which avoids claiming a different hard-voting implementation.

3.6 Joint Optimization Procedure

Given a classical security target λ and a correctness target δ_{corr} , the parameter optimization solves:

$$\min_{w_{r_1}, w_{r_2}, w_e, n_1, \text{mult}, Q, \tau} \text{ct_size}(n_1, \text{mult}, Q) \quad (69)$$

subject to:

$$\text{DFR}_{\text{NSS}}(w_{r_1}, w_{r_2}, w_e, n_1, \text{mult}, Q, \tau) < 2^{-\delta_{\text{corr}}}, \quad (70)$$

$$\text{ISD_cost}(n, w_{\text{sk}}) > 2^\lambda, \quad (71)$$

$$\min(w_{r_1}, w_{r_2}) \geq w_{\text{min}} \quad (\text{ephemeral DQCSD security}), \quad (72)$$

$$Q \in \{2, 4, 8\}, \quad \text{mult} = 5. \quad (73)$$

The optimization is performed by exhaustive grid search over the discrete parameter space. For each candidate $(w_{r_1}, w_{r_2}, w_e, Q)$, the DFR is evaluated via Theorem 9, and the minimum n_1 satisfying the DFR constraint is selected. The parameter table uses $\delta_{\text{corr}} = 128$, matching the conventional HQC/FO correctness target. If a deployment profile requires the decryption-failure term $Q_D \cdot \text{DFR}_{\text{NSS}}$ to be negligible at a higher query budget or higher claimed CCA margin, the same search procedure can be rerun with a larger δ_{corr} .

4 Security Analysis

This section provides a comprehensive security analysis of NSS-HQC. We first formalize the underlying computational assumptions used by the PKE layer (Section 4.1), then present a game-hopping security proof reducing IND-CCA2 security to these assumptions (Section 4.2), discuss the advantage loss terms (Section 4.3), analyze all known attack vectors with concrete cost formulas (Section 4.4), and finally verify the concrete security strength for each parameter set against ISD attacks (Section 4.5).

4.1 Hard Problems Underlying NSS-HQC

The security of NSS-HQC rests on the standard HQC family of quasi-cyclic syndrome-decoding indistinguishability assumptions. The public key uses the usual DQCSD distribution, while the PKE encryption proof additionally uses a joint PKE-core distribution that keeps the correlation between the first ciphertext component and the projected second-component mask. We formalize these assumptions and their relationship to the standard Syndrome Decoding (SD) problem below.

Definition 10 (Syndrome Decoding Problem, $\text{SD}(N, K, t)$). Given a uniformly random parity-check matrix $\mathbf{H} \xleftarrow{\$} \mathbb{F}_2^{(N-K) \times N}$ and a syndrome $\mathbf{s} = \mathbf{H}\mathbf{e}^T$ where $\mathbf{e} \xleftarrow{\$} \{e \in \mathbb{F}_2^N : \text{wt}(e) = t\}$, find $\mathbf{e}$.

The SD problem is known to be NP-complete in the worst case [2]. NSS-HQC uses a structured variant where the parity-check matrix has quasi-cyclic structure, enabling efficient multiplication but potentially providing additional structure to an attacker.

Definition 11 (Generalized Decisional Quasi-Cyclic Syndrome Decoding, $\text{DQCSD}(n, w_x, w_y)$). Let n be a prime with $\text{ord}_n(2) = n - 1$, and let $R = \mathbb{F}_2[X]/(X^n - 1)$. The DQCSD problem is parameterized by (n, w_x, w_y) and defined by the following experiment:

1. The challenger samples $h \xleftarrow{\$} R$ uniformly at random.
2. The challenger samples a bit $b \xleftarrow{\$} \{0, 1\}$.
3. If $b = 0$: the challenger samples $(x, y) \xleftarrow{\$} R_{w_x} \times R_{w_y}$ and computes $s = x + h \cdot y \in R$.
4. If $b = 1$: the challenger samples $s \xleftarrow{\$} R$ uniformly at random.
5. The challenger sends (h, s) to the adversary $\mathcal{A}$.
6. The adversary outputs a guess $b' \in \{0, 1\}$.

The DQCSD advantage of $\mathcal{A}$ is defined as:

$$\text{Adv}_{n, w_x, w_y}^{\text{DQCSD}}(\mathcal{A}) = |\Pr[b' = 1 \mid b = 0] - \Pr[b' = 1 \mid b = 1]|. \quad (74)$$

When $w_x = w_y = w$, we write $\text{DQCSD}(n, w)$ as shorthand for $\text{DQCSD}(n, w, w)$.

The DQCSD problem captures the indistinguishability of the public key $(h, s = x + h \cdot y)$ from a uniformly random pair (h, s') and underlies the standard HQC key-recovery hardness analysis. NSS-HQC keeps this public-key distribution, while the encryption challenge itself is handled by the JPCI assumption below.

Definition 12 (Joint PKE-core indistinguishability, JPCI). For a public key $\text{pk} = (\text{seed}_{\text{PKE}}, s)$ generated as in $\text{NSS-HQC-PKE.KeyGen}$, let $h = \text{Expand}(\text{seed}_{\text{PKE}} \parallel "h", n)$ and sample $r_1 \xleftarrow{\$} R_{w_{r_1}}$, $r_2 \xleftarrow{\$} R_{w_{r_2}}$, and $e \xleftarrow{\$} R_{w_e}$. The JPCI problem asks to distinguish the correlated PKE-core pair

$$(u, z) = (r_1 + h \cdot r_2, \pi_{n_C}(s \cdot r_2 + e))$$

from a uniformly random pair $(u, z) \xleftarrow{\$} R \times \mathbb{F}_2^{n_C}$, given (pk, u, z) . We denote the corresponding distinguishing advantage by $\text{Adv}_{n, n_C, w_{\text{sk}}, w_{r_1}, w_{r_2}, w_e}^{\text{JPCI}}(\mathcal{A})$.

This joint formulation is essential because the same ephemeral vector r_2 appears in both u and z . Treating u and z as two independent game hops would not faithfully model the implemented encryption algorithm.

Assumption 13 (DQCSD-family and joint PKE-core hardness). For every probabilistic polynomial-time adversary $\mathcal{A}$, the advantages $\text{Adv}_{n, w_{\text{sk}}, w_{\text{sk}}}^{\text{DQCSD}}(\mathcal{A})$, $\text{Adv}_{n, w_{r_1}, w_{r_2}}^{\text{DQCSD}}(\mathcal{A})$, and $\text{Adv}_{n, n_C, w_{\text{sk}}, w_{r_1}, w_{r_2}, w_e}^{\text{JPCI}}(\mathcal{A})$ are negligible for the parameter sets in Table 1. Concretely, the best known attacks are ISD-type attacks against the corresponding key-recovery and message-recovery syndrome-decoding instances; see Sections 4.5 and 4.4.5.

Remark 14. The public key algebraic structure of NSS-HQC is identical to that of the standard HQC specification [11]: $\text{pk} = (\text{seed}_{\text{PKE}}, s)$ with $s = x + h \cdot y$, and the encapsulation key distribution is the same. The secret key format differs (NSS-HQC stores $\text{sk} = (\text{seed}_{\text{sk}}, \sigma, \text{pk})$ with on-demand re-derivation of (x, y) via $I(\text{seed}_{\text{sk}})$), but this does not affect the public key distribution. The modifications introduced by NSS-HQC (asymmetric ephemeral weights, ciphertext quantization, and projection onto the codeword support) affect the encryption and decryption procedures; this is why the PKE proof uses the joint JPCI distribution for the correlated pair $(u, \pi_{n_C}(s \cdot r_2 + e))$. Furthermore, the NP-completeness of decoding quasi-cyclic codes [2] provides theoretical support for the worst-case hardness of the underlying search problem.

4.2 Security Proof

We prove the IND-CCA2 security of NSS-HQC-KEM through a modular approach: we first establish the IND-CPA security of the underlying PKE scheme (Lemma 16), and then lift it to IND-CCA2 security via the Fujisaki–Okamoto transform (Theorem 18).

Remark 15 (Implementation prerequisites for the security proof). *The security proof relies on the following three properties of the NSS-HQC implementation, all of which are satisfied by the specification in Section 2:*

1. **Dither independence.** The dither vector d is derived from $\text{salt} \parallel \text{pk}$ (Section 2.7.3), and is independent of the encryption randomness θ and the plaintext m . This ensures that the adversary gains no IND advantage from observing the dither, since salt and pk are already public.
2. **Deterministic quantization.** The compression map $\hat{v} = \text{CompressBlockwise}(v_{\text{raw}} \oplus d)$ is a deterministic function of v_{raw} and d (Section 2.7.1). No additional randomness is introduced by the quantization step, so the ciphertext distribution is fully determined by $(m, \theta, \text{salt}, \text{pk})$.
3. **Fresh salt per encapsulation.** The salt $\text{salt} \xleftarrow{\$} \{0, 1\}^{\lambda_{\text{salt}}}$ is sampled uniformly at random independently for each encapsulation (Algorithm 5, Step 1). This provides domain separation across sessions and prevents multi-target attacks on the hash function G .

4.2.1 IND-CPA Security of NSS-HQC-PKE

Lemma 16 (IND-CPA security). *For any IND-CPA adversary $\mathcal{A}$ against NSS-HQC-PKE that runs in time T , there exists an adversary $\mathcal{B}$ against the JPCI distribution running in time $T + O(n \log n)$ such that:*

$$\text{Adv}_{\text{NSS-HQC-PKE}}^{\text{IND-CPA}}(\mathcal{A}) \leq 2 \text{Adv}_{n, n_C, w_{\text{sk}}, w_{r_1}, w_{r_2}, w_e}^{\text{JPCI}}(\mathcal{B}). \quad (75)$$

Proof. We proceed by a two-game argument. Let Win_i denote the event that the adversary outputs $b' = b$ in Game i .

Game 0 (Real IND-CPA experiment). This is the standard IND-CPA security game:

1. The challenger runs KeyGen to obtain (pk, sk) , where $\text{pk} = (\text{seed}_{\text{PKE}}, s)$ with $s = x + h \cdot y$.
2. The challenger gives pk to $\mathcal{A}$.
3. $\mathcal{A}$ submits two messages $m_0, m_1 \in \{0, 1\}^{8k_1}$.
4. The challenger samples $b \xleftarrow{\$} \{0, 1\}$, $\theta \xleftarrow{\$} \{0, 1\}^{512}$, and $\text{salt} \xleftarrow{\$} \{0, 1\}^{\lambda_{\text{salt}}}$, computes $\text{ct}^* \leftarrow \text{Encrypt}(\text{pk}, m_b, \theta, \text{salt})$, and returns ct^* to $\mathcal{A}$.

5. $\mathcal{A}$ outputs b' .

By definition:

$$\text{Adv}^{\text{IND-CPA}}(\mathcal{A}) = |2\Pr[\text{Win}_0] - 1|. \quad (76)$$

Game 1 (Replace the correlated PKE-core pair). Game 1 is identical to Game 0, except that the correlated pair

$$(u^*, z^*) = (r_1 + h \cdot r_2, \pi_{n_C}(s \cdot r_2 + e))$$

is replaced by a uniformly random pair $(u^*, z^*) \xleftarrow{\$} R \times \mathbb{F}_2^{n_C}$. The challenge ciphertext is then formed as

$$v_{\text{raw}}^* = C.\text{Encode}(m_b) + z^*, \quad \hat{v}^* = Q.\text{CompressBlockwise}(v_{\text{raw}}^* \oplus d).$$

We construct a JPCI distinguisher $\mathcal{B}$ as follows. $\mathcal{B}$ receives a challenge (pk, u, z) and must decide whether (u, z) is the real PKE-core pair with the shared ephemeral vector r_2 , or a uniformly random pair. $\mathcal{B}$ proceeds by:

1. Giving pk to $\mathcal{A}$ and receiving challenge messages m_0, m_1 .
2. Sampling $b \xleftarrow{\$} \{0, 1\}$ and salt $\xleftarrow{\$} \{0, 1\}^{\lambda_{\text{salt}}}$.
3. Computing d from salt $\parallel \text{pk}$. Then set

$$v_{\text{raw}} = C.\text{Encode}(m_b) + z, \quad \hat{v} = Q.\text{CompressBlockwise}(v_{\text{raw}} \oplus d).$$

4. Returning the challenge ciphertext $(u, \hat{v})$ to $\mathcal{A}$.
5. Outputting $b' \oplus b$ (i.e., 1 if $\mathcal{A}$ guesses correctly, 0 otherwise).

If the challenge is structured, this perfectly simulates Game 0. If the challenge is random, this simulates Game 1. Therefore:

$$|\Pr[\text{Win}_0] - \Pr[\text{Win}_1]| \leq \text{Adv}_{n, n_C, w_{\text{sk}}, w_{r_1}, w_{r_2}, w_e}^{\text{JPCI}}(\mathcal{B}). \quad (77)$$

Analysis of Game 1. In Game 1, z^* is uniformly random in $\mathbb{F}_2^{n_C}$. Therefore $v_{\text{raw}}^* = C.\text{Encode}(m_b) + z^*$ is uniformly random and independent of b . Since d is public and deterministic from (salt, pk), $v_{\text{raw}}^* \oplus d$ is also uniform, and $\hat{v}^*$ is a deterministic function of this uniform input. Thus the entire challenge ciphertext $\text{ct}^* = (u^*, \hat{v}^*)$ is independent of the challenge bit:

$$\Pr[\text{Win}_1] = \frac{1}{2}. \quad (78)$$

Combining. From Eqs. (77) and (78):

$$\begin{aligned} \text{Adv}^{\text{IND-CPA}}(\mathcal{A}) &= |2\Pr[\text{Win}_0] - 1| = 2|\Pr[\text{Win}_0] - 1/2| \\ &\leq 2|\Pr[\text{Win}_0] - \Pr[\text{Win}_1]| \\ &\leq 2\text{Adv}_{n, n_C, w_{\text{sk}}, w_{r_1}, w_{r_2}, w_e}^{\text{JPCI}}(\mathcal{B}), \end{aligned} \quad (79)$$

which is Eq. (75). This completes the proof. $\square$

Remark 17. The proof above keeps the asymmetric weights and the shared r_2 explicit. The first ciphertext component has $\text{DQCSD}(n, w_{r_1}, w_{r_2})$ as its marginal distribution, but IND-CPA security is reduced to JPCI because the same r_2 also appears in the projected second-component mask $\pi_{n_C}(s \cdot r_2 + e)$. The concrete ISD costs for the corresponding key-recovery, ephemeral, and message-recovery distributions are checked in the later attack analysis.

4.2.2 From IND-CPA to IND-CCA2 via the FO Transform

The KEM is obtained from NSS-HQC-PKE by applying the $\text{FO}_m^\perp$ transform of Hofheinz, Hövelmanns, and Kiltz [12]. This transform converts any δ -correct IND-CPA-secure PKE into an IND-CCA2-secure KEM in the (quantum) random oracle model, where $\delta = \text{DFR}_{\text{NSS}}$ is the correctness error.

The transform works as follows. Let $\text{PKE} = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt})$ be the underlying PKE scheme. The KEM derived via $\text{FO}_m^\perp$ has:

- **Encapsulation:** Sample $m \xleftarrow{\$} \mathcal{M}$ and salt, derive $\theta = G(m \parallel \text{salt} \parallel \text{pk})$, compute $\text{ct} = \text{Encrypt}(\text{pk}, m, \theta, \text{salt})$, set $\text{ct}_{\text{full}} = \text{ct} \parallel \text{salt}$, and output $K = H_\kappa(m \parallel \text{ct}_{\text{full}})$.
- **Decapsulation:** Parse ct_{full} as (ct, salt) , decrypt to get m' , re-derive $\theta' = G(m' \parallel \text{salt} \parallel \text{pk})$, and re-encrypt to get ct' . If $\text{ct}' = \text{ct}$, output $K = H_\kappa(m' \parallel \text{ct}_{\text{full}})$; otherwise output $K = H_\kappa(\sigma \parallel \text{ct}_{\text{full}})$ (implicit rejection using the secret σ).

Theorem 18 (IND-CCA2 security of NSS-HQC-KEM). *In the quantum random oracle model (QROM), for any IND-CCA2 adversary $\mathcal{A}$ against NSS-HQC-KEM that makes at most Q_D classical decapsulation queries and at most Q_H quantum random-oracle queries, there exists an adversary $\mathcal{B}$ against the JPCI assumption such that:*

$$\text{Adv}_{\text{NSS-HQC-KEM}}^{\text{IND-CCA2}}(\mathcal{A}) \leq 4 \text{Adv}_{n, n_C, w_{\text{sk}}, w_{r_1}, w_{r_2}, w_e}^{\text{JPCI}}(\mathcal{B}) + 2Q_D \cdot \text{DFR}_{\text{NSS}} + 2\varepsilon_{\text{FO}}, \quad (80)$$

where the FO loss term is bounded by:

$$\varepsilon_{\text{FO}} \leq \frac{(Q_H + Q_D)^2}{|\mathcal{M}|} + \frac{(Q_H + Q_D) \cdot Q_D}{2^{\lambda_{\text{salt}}}}, \quad (81)$$

with $|\mathcal{M}| = 2^{8k_1}$ being the message space size (e.g., $|\mathcal{M}| = 2^{128}$ for NSS-HQC-128 with $k_1 = 16$).

Proof. We apply the modular FO analysis of [12] (Theorem 3.4 therein) to NSS-HQC-PKE. The proof proceeds through four games.

Game 0 (Real IND-CCA2 experiment). The real KEM security game with adversary $\mathcal{A}$. The challenger generates keys, encapsulates to produce $(K_0^*, \text{ct}_{\text{full}}^*)$, samples $K_1^* \xleftarrow{\$} \{0, 1\}^\kappa$, flips $b \xleftarrow{\$} \{0, 1\}$, gives $(K_b^*, \text{ct}_{\text{full}}^*)$ to $\mathcal{A}$, and answers decapsulation queries. $\mathcal{A}$ outputs b' .

$$\text{Adv}^{\text{IND-CCA2}}(\mathcal{A}) = |2 \Pr[\text{Win}_0] - 1|. \quad (82)$$

Game 1 (Simulate decapsulation via random oracle). Game 1 replaces the decapsulation oracle with a simulated oracle. For each decapsulation query ct_{full} , the game parses it as (ct, salt) and searches the random-oracle table for an entry (m, salt) such that $\theta = G(m \parallel \text{salt} \parallel \text{pk})$ and $\text{Encrypt}(\text{pk}, m, \theta, \text{salt}) = \text{ct}$. If found, it returns $H_\kappa(m \parallel \text{ct}_{\text{full}})$; otherwise it returns $H_\kappa(\sigma \parallel \text{ct}_{\text{full}})$.

The simulation fails only when a decapsulation query ct_{full} is valid but the corresponding (m, salt) was never queried to the random oracle. This occurs with probability at most:

$$|\Pr[\text{Win}_0] - \Pr[\text{Win}_1]| \leq \frac{Q_D}{|\mathcal{M}|}. \quad (83)$$

Game 2 (Handle decryption failures). Game 2 is identical to Game 1, except that whenever a decapsulation query triggers a decryption failure (i.e., Decrypt returns an incorrect $m' \neq m$), the game aborts. Since each decapsulation query fails with probability at most DFR_{NSS} :

$$|\Pr[\text{Win}_1] - \Pr[\text{Win}_2]| \leq Q_D \cdot \text{DFR}_{\text{NSS}}. \quad (84)$$

Game 3 (Replace K_0^* with random). Game 3 replaces the real shared secret $K_0^* = H_\kappa(m^* \parallel \text{ct}_{\text{full}}^*)$ with a uniformly random value. In the QROM, this requires reprogramming the random oracle H_κ at one point, which the adversary can detect only if it queries H_κ at exactly $(m^* \parallel \text{ct}_{\text{full}}^*)$. Since m^* is uniformly random and independent of the adversary's view in Game 2 (by the IND-CPA security of the PKE), this probability is bounded by:

$$|\Pr[\text{Win}_2] - \Pr[\text{Win}_3]| \leq \text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{B}') + \frac{(Q_H + Q_D)^2}{|\mathcal{M}|}. \quad (85)$$

In Game 3, the challenge key K_b^* is uniformly random regardless of b , so $\Pr[\text{Win}_3] = 1/2$.

Combining.

$$\begin{aligned} \text{Adv}^{\text{IND-CCA2}}(\mathcal{A}) &\leq 2(\text{Adv}^{\text{IND-CPA}}(\mathcal{B}') + Q_D \cdot \text{DFR}_{\text{NSS}} + \varepsilon_{\text{FO}}) \\ &\leq 4 \text{Adv}_{n, n_C, w_{\text{sk}}, w_{r_1}, w_{r_2}, w_e}^{\text{JPCI}}(\mathcal{B}) + 2Q_D \cdot \text{DFR}_{\text{NSS}} + 2\varepsilon_{\text{FO}}, \end{aligned} \quad (86)$$

where the last step applies Lemma 16. The salt collision term $(Q_H + Q_D) \cdot Q_D / 2^{\lambda_{\text{salt}}}$ accounts for the probability that two encapsulations use the same salt. The Game 0 $\rightarrow$ 1 gap $Q_D / |\mathcal{M}|$ from Eq. (83) does not appear as a separate term because it is dominated by the message-space collision term $(Q_H + Q_D)^2 / |\mathcal{M}|$ already present in ε_{FO} : since $Q_H + Q_D \geq Q_D$ and $Q_D \geq 1$, we have $(Q_H + Q_D)^2 \geq Q_D$, so the larger term subsumes the smaller. $\square$

4.3 Discussion of the Advantage Loss

The security bound in Theorem 18 contains three loss terms. We discuss each in detail.

(1) PKE-core indistinguishability loss. This is the computational core of the security guarantee. It is captured by the JPCI assumption, which jointly handles the first ciphertext component $u = r_1 + h \cdot r_2$ and the projected mask $\pi_{n_C}(s \cdot r_2 + e)$ in v_{raw} with the same ephemeral vector r_2 . The related key-recovery, ephemeral, and message-recovery costs are analyzed in Section 4.5 using information set decoding (ISD) algorithms.

(2) Decryption failure loss: $Q_D \cdot \text{DFR}_{\text{NSS}}$. Each decapsulation query may trigger a decryption failure with probability DFR_{NSS} , and each failure potentially leaks one bit of information about the secret key through the failure/success distinction. With the current correctness target $\text{DFR}_{\text{NSS}} < 2^{-128}$ and a polynomial number of queries $Q_D < 2^{64}$, this term is bounded by $Q_D \cdot 2^{-128} < 2^{-64}$. This establishes a 128-bit correctness margin for the submitted parameter table. If a higher CCA margin is required for a higher security profile, the optimization in Section 3 can be rerun with a larger δ_{corr} . The DFR bound is enforced by the unified analysis in Section 3.

(3) FO transform loss: ε_{FO} . This term has two sub-components:

- *Message-space collision:* $(Q_H + Q_D)^2 / |\mathcal{M}|$. For $|\mathcal{M}| = 2^{8k_1}$ (e.g., 2^{128} for NSS-HQC-128), this term must be included explicitly in any concrete query budget. For example, if $Q_H + Q_D < 2^{48}$, the term is bounded by 2^{-32} ; this is acceptable only for a correspondingly bounded concrete attack model, not as a standalone 128-bit loss term. For higher security levels ($k_1 \geq 32$, giving $|\mathcal{M}| \geq 2^{256}$), the bound is significantly tighter.
- *Salt collision:* $(Q_H + Q_D) \cdot Q_D / 2^{\lambda_{\text{salt}}}$. For $\lambda_{\text{salt}} = 256$ and $Q_H, Q_D < 2^{64}$, this is $< 2^{128} / 2^{256} = 2^{-128}$.

Both sub-components must be evaluated together with the intended query budget. The security tables below therefore report the algebraic decoding strength separately from the FO and DFR proof-loss terms.

4.4 Analysis of Known Attacks

We classify known attacks into five categories, each targeting a different component of the security proof.

4.4.1 Key-Recovery Attacks (targeting DQCSD)

Information Set Decoding (ISD). The best known classical attack on the key-recovery problem is ISD. Given the public key (h, s) with $s = x + h \cdot y$, key recovery is equivalent to solving the syndrome decoding problem $\text{SD}(2n, n, 2w_{\text{sk}})$: find two vectors $x, y \in R_{w_{\text{sk}}}$ such that $x + h \cdot y = s$. By concatenating x and y into a single vector of length $2n$ and weight $2w_{\text{sk}}$, this becomes a standard SD instance with parity-check matrix $[\mathbf{I}_n \mid \mathbf{H}_h]$ where $\mathbf{H}_h$ is the circulant matrix defined by h .

The main ISD algorithms and their asymptotic complexities for $\text{SD}(N, K, t)$ are:

- **Prange (1962):** $T_{\text{Prange}} = \binom{N}{t} / \binom{K}{t}$.
- **Stern (1988):** Improves over Prange by splitting the information set and using birthday-type matching. Complexity $\tilde{O}(2^{c_{\text{Stern}} \cdot N})$ for a constant $c_{\text{Stern}} < c_{\text{Prange}}$.
- **MMT (2011):** Applies the representation technique to Stern’s algorithm.
- **BJMM (2012):** Further improves MMT using nearest-neighbor search. Currently the best known classical ISD algorithm for the HQC weight regime.
- **May–Ozerov (2015):** Uses locality-sensitive hashing for nearest-neighbor search; asymptotically better than BJMM but practically comparable.

For the HQC key-recovery instance, where $N = 2n$, $K = n$, and $t = 2w_{\text{sk}}$ with $w_{\text{sk}} \approx 0.5\sqrt{n}$, the BJMM cost under a 2^{60} -bit memory constraint is estimated by the scaling formula derived in Section 4.5.

Algebraic attacks. Beyond ISD, algebraic attacks attempt to exploit the quasi-cyclic structure of the code. However, no algebraic attack is currently known to outperform ISD for the HQC parameters [11, 14].

4.4.2 Message-Recovery Attacks (targeting ciphertext)

Given a ciphertext $(u, \hat{v})$, an attacker may attempt to recover the ephemeral vectors (r_1, r_2, e) without the secret key. For the underlying uncompressed HQC core, the natural decoding model is the syndrome-decoding instance $\text{SD}(3n, n, w_{r_1} + w_{r_2} + w_e)$. Because NSS-HQC publishes only the quantized value $\hat{v}$ rather than the full second component, this model should be read as the conservative ISD estimator used for the PKE core, not as an exact reduction from the compressed ciphertext.

For the selected parameters, the estimator gives:

$$T_{\text{msg}}^{\text{cl}} \geq T_{\text{key}}^{\text{cl}}, \quad (87)$$

so message recovery is not the limiting algebraic attack in Table 7. A submission package should report the estimator output for this term, rather than relying only on the informal observation that the modeled instance has a longer ambient length and larger error weight.

Effect of quantization. The ciphertext compression in NSS-HQC transmits

$$\hat{v} = Q.\text{CompressBlockwise}(v \oplus d)$$

instead of the full v . Since this step is deterministic given the public dither, the proof does not treat compression as an independent source of hardness. Security is instead tied to the JPCI assumption and the corresponding decoding-cost estimates for the underlying PKE-core distribution.

4.4.3 Decryption Failure Attacks

An adversary may craft ciphertexts designed to trigger decryption failures with elevated probability, exploiting the failure event to extract information about the secret key. The key defense is ensuring a sufficiently small DFR_{NSS} : with the submitted bound $\text{DFR}_{\text{NSS}} < 2^{-128}$, even after 2^{64} adaptive decapsulation queries, the expected number of natural failures is below 2^{-64} under the modeled channel. Higher CCA proof margins require correspondingly stronger DFR estimates.

The concrete DFR bound is established by the unified analysis in Section 3, which accounts for both the projected structural error e'_{struct} and the quantization error proxy $\hat{e}_{\text{quant}}$ through the BSC cascade formula.

4.4.4 Seed and Target Attacks

Secret-seed and rejection-secret exhaustion. An adversary may attempt to guess the secret key seed seed_{sk} or the implicit-rejection secret σ . These are the secret values stored in sk ; the public seed seed_{PKE} is not a target because it is transmitted as part of the public key. The exhaustive-search cost for seed_{sk} is:

$$T_{\text{seed}}^{\text{cl}} = 2^{\lambda_{\text{seed}}-1}, \quad T_{\text{seed}}^{\text{qu}} = 2^{\lambda_{\text{seed}}/2} \quad (\text{Grover search}). \quad (88)$$

The same calculation applies to σ with κ in place of λ_{seed} . For $\lambda_{\text{seed}} \geq 256$ and $\kappa \geq 256$, both costs exceed the 128-bit classical target and are included in the per-level security accounting below.

Target-key guessing and hash preimage attacks. A direct attack on the shared-secret output requires guessing the κ -bit key or finding the relevant random-oracle preimage. The classical cost is 2^κ , and Grover search gives quantum cost $2^{\kappa/2}$. Collision resistance of the output space is not the limiting notion for KEM key indistinguishability; the FO proof instead accounts for message-space and salt-collision losses explicitly in Eq. (81).

4.4.5 Asymmetric Weight Security

NSS-HQC uses asymmetric ephemeral weights $w_{r_1} = w_{r_2} < w_{\text{sk}}$. The ciphertext component $u = r_1 + h \cdot r_2$ defines an ephemeral DQCSD instance with weights (w_{r_1}, w_{r_2}) . We must verify that this instance remains hard.

In the HQC weight regime $w = \Theta(\sqrt{n})$, the ISD cost depends primarily on the code length n and is only weakly sensitive to moderate weight changes. Specifically, the BJMM cost for $\text{SD}(2n, n, 2w)$ scales approximately as $e^{c \cdot w \cdot \log(n/w)}$ for a constant c . A 10% reduction in w (from w_{sk} to $\min(w_{r_1}, w_{r_2}) \approx 0.9w_{\text{sk}}$) changes the exponent by approximately:

$$\Delta T \approx 0.1 \cdot w_{\text{sk}} \cdot \log(n/w_{\text{sk}}) \cdot c, \quad (89)$$

which is a small fraction of the total ISD cost $T_{\text{key}}^{\text{cl}}$. For all four NSS-HQC parameter sets, the weight reduction is at most 10% (see Table 5), and the ISD cost remains well above the security target.

Table 5. Ephemeral weight reduction analysis.

Parameter set	$\min(w_{r_1}, w_{r_2})$	w_{sk}	Reduction	ISD impact
NSS-HQC-128	66	73	9.6%	< 3 bits
NSS-HQC-256	106	117	9.4%	< 5 bits
NSS-HQC-384	159	175	9.1%	< 7 bits
NSS-HQC-512	213	233	8.6%	< 9 bits

Since T_{key}^{cl} exceeds the classical target by at least 32 bits for NSS-HQC-128 and remains on target for other levels, the ISD cost reduction from asymmetric weights is absorbed by the existing security margin.

4.5 Security Strength Analysis

4.5.1 Methodology

We separate the *algebraic decoding work factor* from the *CCA proof-loss terms*. The algebraic work factor is the minimum bit-complexity across the known decoding, seed-search, and generic target attacks:

$$\text{AlgebraicLevel} = \min(T_{key}, T_{msg}, T_{seed}, T_{target}), \quad (90)$$

where each term is defined as follows:

- T_{key} : Key-recovery cost via ISD on $SD(2n, n, 2w_{sk})$. This is the dominant term for the DQCSD assumption.
- T_{msg} : Message-recovery cost estimated via ISD on the modeled PKE-core instance with length $3n$, dimension n , and weight $w_{r_1} + w_{r_2} + w_e$.
- $T_{seed} = \lambda_{seed} - 1$: Exhaustive search over the seed.
- T_{target} : Direct target-key guessing or random-oracle preimage search on the κ -bit shared-secret output.

For the classical algebraic strength, we report $\min(T_{key}^{cl}, T_{msg}^{cl}, T_{seed})$. For the quantum algebraic strength, we report $\min(T_{key}^{qu}, T_{seed}^{qu}, T_{target}^{qu})$, where the quantum costs account for Grover-type speedups. The decryption-failure term is not an ISD attack cost; it enters the CCA2 bound separately as $Q_D \cdot \text{DFR}_{NSS}$. The submitted parameter table uses $\text{DFR}_{NSS} < 2^{-128}$, giving a 128-bit correctness margin unless the grid search is rerun with a larger δ_{corr} .

4.5.2 ISD Cost Estimation

The classical ISD cost for the key-recovery instance $SD(2n, n, 2w_{sk})$ is estimated using a calibrated scaling formula. In the HQC weight regime where $w_{sk} \approx 0.5\sqrt{n}$, the BJMM algorithm under a 2^{60} -bit memory constraint yields a bit-complexity that scales as:

$$T_{key}^{cl} \approx c \cdot \sqrt{n}, \quad c = \frac{T_{ref}}{\sqrt{n_{ref}}}. \quad (91)$$

Calibration. The constant c is determined from the known data point for the standard HQC-128 parameters: $T_{ref} = 145.80$ bits at $n_{ref} = 17,669$, as reported in the ISD analysis of [4] using

the CryptographicEstimators library [13]. This yields:

$$c = \frac{145.80}{\sqrt{17,669}} = \frac{145.80}{132.93} = 1.0969. \quad (92)$$

Cross-validation. The formula is validated against two additional known data points:

Table 6. Cross-validation of the ISD scaling formula against known HQC instances.

Instance	n	Predicted T_{cl}	NIST target	Margin
HQC-128	17,669	145.8	≥ 128	+17.8
HQC-192	35,851	207.7	≥ 192	+15.7
HQC-256	57,637	263.3	≥ 256	+7.3

All three predictions exceed their NIST security targets with consistent margins, supporting the validity of the scaling formula for the HQC weight regime.

Quantum ISD cost. The best known quantum ISD algorithm is quantum Prange [15], which applies Grover’s search to accelerate the permutation-guessing phase of classical Prange. The quantum cost satisfies:

$$T_{\text{key}}^{\text{qu}} \approx \frac{T_{\text{key}}^{\text{cl}}}{2}. \quad (93)$$

This is a conservative estimate. The actual quantum speedup is smaller than a full square root because the dominant cost components of advanced ISD algorithms (Gaussian elimination, nearest-neighbor search) are not amenable to Grover acceleration [15]. The factor-of-2 estimate corresponds to the most favorable case for the attacker and is adopted as a conservative security bound.

4.5.3 Per-Parameter-Set Analysis

We apply the methodology above to each of the four NSS-HQC parameter sets.

NSS-HQC-128. Key recovery: $\text{SD}(58,886, 29,443, 146)$. Classical ISD: $T_{\text{key}}^{\text{cl}} = 1.0969 \times \sqrt{29,443} \approx 188.2$ bits. Quantum ISD: $T_{\text{key}}^{\text{qu}} \geq 94.1$ bits. Message recovery: $\text{SD}(88,329, 29,443, 249)$ where $249 = 66 + 66 + 117$; estimated $T_{\text{msg}}^{\text{cl}} \geq 191$ bits under the PKE-core model. Seed: $T_{\text{seed}} = 256$. Algebraic classical strength is at least 188 bits, while the submitted CCA correctness margin is 128 bits through $\text{DFR}_{\text{NSS}} < 2^{-128}$. The raw quantum algebraic estimate is at least 94.1 bits, exceeding the 80-bit quantum target associated with the 128-bit classical category.

NSS-HQC-256. $T_{\text{key}}^{\text{cl}} = 1.0969 \times \sqrt{54,493} \approx 256.0$ bits. $T_{\text{key}}^{\text{qu}} = 128.0$ bits.

NSS-HQC-384. $T_{\text{key}}^{\text{cl}} = 1.0969 \times \sqrt{122,579} \approx 384.0$ bits. $T_{\text{key}}^{\text{qu}} = 192.0$ bits.

NSS-HQC-512. $T_{\text{key}}^{\text{cl}} = 1.0969 \times \sqrt{217,901} \approx 512.0$ bits. $T_{\text{key}}^{\text{qu}} = 256.0$ bits.

Table 7. Security strength analysis for all four NSS-HQC parameter sets.

Set	SD instance	$T_{\text{key}}^{\text{cl}}$	$T_{\text{key}}^{\text{qu}}$	$T_{\text{msg}}^{\text{cl}}$	DFR margin	T_{seed}	Cl. alg.	Qu. alg.
128	SD(58886,29443,146)	188.2	≥ 94.1	≥ 191	128	256	$\geq \mathbf{128}$	$\geq \mathbf{94.1}$
256	SD(108986,54493,234)	256.0	128.0	≥ 260	128	256	$\geq \mathbf{256}$	$\geq \mathbf{128}$
384	SD(245158,122579,350)	384.0	192.0	≥ 390	128	384	$\geq \mathbf{384}$	$\geq \mathbf{192}$
512	SD(435802,217901,466)	512.0	256.0	≥ 519	128	512	$\geq \mathbf{512}$	$\geq \mathbf{256}$

4.5.4 Compliance Summary

Table 8. Algebraic compliance verification against the submission security requirements. The CCA proof also includes the DFR and FO loss terms discussed above.

Set	Cl. req.	Cl. achieved	Pass	Qu. req.	Qu. achieved	Pass
128	≥ 128	188.2	✓	≥ 80	≥ 94.1	✓
256	≥ 256	256.0	✓	≥ 128	128.0	✓
384	≥ 384	384.0	✓	≥ 192	192.0	✓
512	≥ 512	512.0	✓	≥ 256	256.0	✓

The table reports the decoding and seed-search work factors. The CCA2 theorem additionally contains the terms $2Q_D \cdot \text{DFR}_{\text{NSS}}$ and $2\epsilon_{\text{FO}}$. Therefore, with the currently selected $\delta_{\text{corr}} = 128$, the manuscript establishes a 128-bit decryption-failure margin for all parameter sets. Claims of 256-, 384-, or 512-bit *CCA proof margin* require rerunning the DFR grid search with a larger δ_{corr} and evaluating the FO loss under the intended query budget.

Remark 19. *The ISD cost estimates use the scaling formula $T_{\text{cl}} \approx 1.0969 \cdot \sqrt{n}$, calibrated from HQC-128 and cross-validated against HQC-192 and HQC-256. For NSS-HQC-384 and NSS-HQC-512, which involve larger n values than any previously analyzed HQC instance, the accompanying reproducibility material should report the corresponding `CryptographicEstimators` schedules. The n values in Table 1 are the selected primitive primes with $\text{ord}_n(2) = n - 1$.*

4.5.5 Other Security Properties

Session-key independence and forward-secrecy boundary. NSS-HQC-KEM uses fresh ephemeral randomness (m, salt) in each encapsulation, and the shared secret $K = H_\kappa(m \parallel \text{ct}_{\text{full}})$ is bound to the specific ciphertext. This provides independence between honestly generated sessions. However, a standalone static KEM does *not* provide forward secrecy after compromise of the long-term decapsulation key: an attacker who records an old ciphertext and later obtains dk can decapsulate that ciphertext. Forward secrecy must therefore be provided by the higher-level protocol, for example by using NSS-HQC inside an authenticated ephemeral key-exchange design.

Multi-key security. The IND-CCA2 security proof (Theorem 18) applies independently to each key pair. In the N -user setting, the security degrades by at most an additive factor of N (via a standard hybrid argument), yielding:

$$\text{Adv}_{N\text{-user}}^{\text{IND-CCA2}}(\mathcal{A}) \leq N \cdot \text{Adv}_{1\text{-user}}^{\text{IND-CCA2}}(\mathcal{A}). \quad (94)$$

For $N < 2^{32}$ users, the multi-user loss is at most 32 bits, which is absorbed by the security margins in Table 7.

Side-channel resistance. The implementation of NSS-HQC is designed for constant-time execution:

- Fixed-weight sampling uses Fisher–Yates shuffling with data-independent array accesses.
- Polynomial multiplication uses the same code path regardless of the weight of the operands.
- The implicit rejection in Decaps (Algorithm 6) computes both candidate shared secrets and selects via constant-time conditional move, preventing timing leakage.
- The quantization and dithering operations are simple bitwise arithmetic with no secret-dependent branches.

Absence of quantum-vulnerable components. NSS-HQC uses only the following cryptographic primitives, none of which are vulnerable to known polynomial-time quantum attacks:

- The QCSD problem: no known polynomial-time quantum algorithm. Grover acceleration provides at most a square-root speedup, accounted for in the quantum security analysis (Eq. 93).
- SHA3-512 and SHAKE256: quantum collision resistance $\geq 2^{128}$ (via the BHT bound), quantum preimage resistance $\geq 2^{128}$ (via Grover).
- Bitwise arithmetic (XOR, cyclic shift): trivially quantum-safe.

In particular, NSS-HQC does not use integer factorization, discrete logarithms, elliptic curve groups, or any other primitive known to be broken by Shor’s algorithm.

5 Detailed Performance Analysis

This section provides an artifact-level performance evaluation of NSS-HQC from three perspectives: implementation runtime, transmission and storage sizes, and comparison against rebuilt standard HQC reference libraries under the same external benchmark harness. The implementation data reported here are based on the final core-paper-aligned artifact and the rerun CSV files recorded with the submitted benchmark package.

5.1 Performance Analysis Methodology

Cost model. The computational cost of NSS-HQC is decomposed into six primitive operations, each with a well-defined asymptotic and concrete cost:

Table 9. Primitive operations and their asymptotic costs.

Operation	Description	Asymptotic cost	Dominant in
POLYMUL(a, b)	$a \cdot b \in R$ via schoolbook or FFT	$O(n \log n)$	KG, Enc, Dec
FIXEDWTSAMPLE(w)	Sample $v \xleftarrow{\$} R_w$	$O(n)$	KG, Enc
XOF-EXPAND(ℓ)	SHAKE256 output of ℓ bits	$O(\ell)$	KG, Enc, Dec
RM-FHT-DECODE	Soft accumulation plus FHT on 128 values	$O(128 \text{ mult} + n_2 \log n_2)$	Dec
RS-DECODE(n_1, k_1)	BM + Forney over $\mathbb{F}_{256}$	$O(n_1^2)$	Dec
QUANTIZE/DEQUANTIZE	Blockwise compress/decompress	$O(n_C / \text{mult})$	Enc, Dec

Measurement methodology. The implementation results reported in this section are based on the final core-paper-aligned artifact. Runtime values are reported as median wall-clock times in milliseconds obtained from the benchmark harness under the recorded build environment. The benchmark uses deterministic correctness datasets and separate tamper-rejection trials. The reported values characterize the submitted portable implementation artifact and its comparison harness, rather than architecture-independent cycle counts. The benchmark artifacts include the dataset hash, KAT hashes, build precheck logs, and CSV result files.

The benchmark rerun used deterministic datasets and generated CSV artifacts for runtime, size, correctness, tamper behavior, and standard-HQC comparison. The dataset SHA256 is

CD475DB303464EC89D166EFF9584168339E36D65C6EFE7F9511066F4A873F60E.

The KAT files were regenerated for the final parameterization and matched the recorded SHA256 digests in the artifact files.

5.2 Implementation-Level Operation Profile

The portable implementation follows the specification directly. The final benchmark reports end-to-end key generation, encapsulation, and decapsulation times. It does not separate per-primitive cycle counts. Table 10 therefore summarizes the implementation-level role of each major operation and its expected cost profile.

Table 10. Implementation-level operation profile.

Operation	Role in implementation	Expected cost profile
XOF-EXPAND and sampling	Seed expansion and fixed-weight vector generation	Input-size dependent, repeated across key generation and encryption
POLYMUL	Ring product for public-key and ciphertext operations	Dominant arithmetic component
RM-FHT-DECODE	Block decoding of compressed payload	Repeated over encoded blocks
RS-DECODE	Outer error-and-erasure correction	Depends on level parameters
DITHER and quantization	Compression-side randomized masking and blockwise compression	Lightweight bit-level processing relative to ring arithmetic and decoding

Qualitative observations from the profile:

1. **Polynomial multiplication remains the primary arithmetic target.** The ring products follow the same structural family as standard HQC and remain a natural focus for implementation back ends.
2. **Decapsulation includes verification work.** As in FO-style KEMs, decapsulation includes decryption and a re-encryption check before key selection, so it naturally has a larger implementation footprint than encapsulation.
3. **Compression-specific operations are localized.** Dither generation, quantization, and dequantization are confined to the ciphertext-processing path and are best understood as implementation-level bit-processing components.
4. **Asymmetric weights affect the sampled vectors.** The selected weight distribution changes the sparse operands used during encapsulation, while the benchmarked artifact reports the resulting end-to-end runtime rather than a per-primitive cycle attribution.

5.3 Reference Implementation: All Security Levels

Table 11. Portable implementation median runtime on the benchmark platform.

Parameter set	KeyGen ms	Encaps ms	Decaps ms
NSS-HQC-128	7.836450	14.280550	27.423400
NSS-HQC-256	23.998000	43.798600	76.349600
NSS-HQC-384	82.173450	150.560700	244.585850
NSS-HQC-512	198.644600	364.728050	578.171550

Values are median wall-clock times in milliseconds from the final benchmark rerun. They describe the portable artifact under the recorded harness and should not be interpreted as cycle counts or cross-platform performance guarantees.

5.4 Portable Implementation and Optimization Opportunities

The current artifact reports portable implementation measurements. The implementation structure is compatible with standard optimization directions such as vectorized polynomial arithmetic, batched hash/XOF expansion, and vector-friendly decoding kernels. These optimizations are engineering back-end choices and are therefore discussed as optimization opportunities rather than used as the basis for the measured results in this section.

Reliability extraction. The reliability metric ρ_i (Eq. 13) is obtained from the same FHT coefficient information used for RM decoding. Its implementation cost depends on the selected decoder backend and is therefore treated as part of the decoder engineering profile.

Table 12. Implementation optimization opportunities.

Component	Optimization direction	Expected benefit
POLYMUL	Vectorized ring arithmetic	Reduce dominant arithmetic cost
XOF-EXPAND	Batched SHAKE expansion	Reduce repeated expansion overhead
RM-FHT-DECODE	Vector-friendly block decoding	Improve block-level throughput
QUANTIZE/DEQUANTIZE	Word-level bit packing	Reduce compression overhead

5.5 Transmission and Storage Overhead

5.5.1 Public Key Size

The public key consists of the compressed seed and the syndrome vector. Following the standard HQC convention [11], the seed seed_{PKE} occupies $\lambda_{\text{seed}}/8$ bytes:

$$|\text{pk}| = \lambda_{\text{seed}}/8 + \lceil n/8 \rceil \text{ bytes.} \quad (95)$$

For NSS-HQC-128: $|\text{pk}| = 32 + \lceil 29,443/8 \rceil = 32 + 3,681 = 3,713$ bytes. The public key size is determined by the ring dimension n and the selected seed length.

5.5.2 Secret Key Size

The submitted serialisation stores the secret seed, the rejection secret, and a copy of the public key:

- *Serialized form:* $\text{sk} = \text{seed}_{\text{sk}} \parallel \sigma \parallel \text{pk}$, so $|\text{sk}| = \lambda_{\text{seed}}/8 + \kappa/8 + |\text{pk}|$.
- *Expanded cache form:* an implementation may additionally cache the support sets of (x, y) to save regeneration time, adding about $2w_{\text{sk}} \lceil \log_2 n \rceil / 8$ bytes.

Table 13. Secret key sizes in serialised and optional expanded-cache form (bytes).

Scheme	$ \text{sk} $	$ \text{sk}_{\text{cache}} $	w_{sk}	$\lceil \log_2 n \rceil$
NSS-HQC-128	3,777	4,069	73	15
NSS-HQC-256	6,908	7,376	117	16
NSS-HQC-384	15,467	16,211	175	17
NSS-HQC-512	27,430	28,479	233	18

5.5.3 Ciphertext Size

The PKE ciphertext body consists of the first component u and the quantized second component $\hat{v}$:

$$|ct| = \underbrace{\lceil n/8 \rceil}_{|u|} + \underbrace{\lceil n_1 \cdot n_2 \cdot b/8 \rceil}_{|\hat{v}|} \quad \text{bytes.} \quad (96)$$

The full KEM ciphertext appends the salt, so $|ct_{\text{full}}| = |ct| + \lambda_{\text{salt}}/8$. The key innovation of NSS-HQC is the reduction of the $|\hat{v}|$ term. In the standard HQC specification, the second component is uncompressed:

$$|\hat{v}|_{\text{std}} = \lceil n_1 \cdot n_2 \cdot \text{mult}/8 \rceil \quad \text{bytes.} \quad (97)$$

The NSS-HQC quantization replaces this with:

$$|\hat{v}|_{\text{NSS}} = \lceil n_1 \cdot n_2 \cdot b/8 \rceil \quad \text{bytes,} \quad (98)$$

achieving a compression ratio of b/mult on the v component. For $b = 2$ and $\text{mult} = 5$ (uniform across all four security levels): compression ratio = $2/5$, saving 60% on $|\hat{v}|$.

5.5.4 Detailed Size Computation

Table 14. Detailed ciphertext size decomposition (bytes).

Scheme	$ u $	$ \hat{v} _{\text{std}}$	$ \hat{v} _{\text{NSS}}$	$ \hat{v} $ saving	$ ct _{\text{NSS}}$	Total saving
NSS-HQC-128	3,681	3,680	1,472	−60.0%	5,153	−30.0%
NSS-HQC-256	6,812	5,600	2,240	−60.0%	9,052	−27.1%
NSS-HQC-384	15,323	8,000	3,200	−60.0%	18,523	−20.6%
NSS-HQC-512	27,238	10,400	4,160	−60.0%	31,398	−16.6%

Analysis of the “total saving” column. The total ciphertext saving is smaller than the $|\hat{v}|$ saving because the u component (which is not compressed) accounts for a large and increasing fraction of the ciphertext at higher security levels:

$$\text{Total saving} = \frac{|\hat{v}|_{\text{std}} - |\hat{v}|_{\text{NSS}}}{|u| + |\hat{v}|_{\text{std}}}. \quad (99)$$

For NSS-HQC-128, $|u|/|ct|_{\text{uncompr}} = 3,681/7,361 \approx 50\%$, so the 60% reduction in $|\hat{v}|$ translates to a 30% reduction in total ciphertext. For NSS-HQC-256 through NSS-HQC-512, $|u|$ accounts for a growing fraction of total bandwidth as n increases while $|\hat{v}|_{\text{NSS}}$ grows only with n_1 , yielding progressively smaller total savings. This analysis reveals that **further bandwidth improvements would require compressing the u component**, which is an open research direction beyond the scope of NSS-HQC.

5.6 Complete Size Comparison

Table 15. Final API size comparison across NSS-HQC parameter sets (bytes).

Parameter set	Public key	Secret key	KEM ciphertext	Shared secret	pk + ct
NSS-HQC-128	3,713	3,777	5,185	32	8,898
NSS-HQC-256	6,844	6,908	9,084	32	15,928
NSS-HQC-384	15,371	15,467	18,571	48	33,942
NSS-HQC-512	27,302	27,430	31,462	64	58,764

KEM ciphertext denotes the full KEM ciphertext $ct_{full} = ct \parallel salt$. The secret-key column reports the complete serialized decapsulation key used by the final API, not only the seed material.

5.7 Comparison with Standard HQC

The comparison should be interpreted as an artifact-level benchmark comparison under a common external harness. The communication figures follow the final API sizes, while the run-time figures are median wall-clock measurements from the rerun. They are not intended as architecture-independent cycle counts. The standard HQC comparison uses rebuilt official HQC core libraries evaluated through the same external harness.

Table 16. NSS-HQC and standard HQC artifact size comparison (bytes).

Scheme	Level	pk	sk	ct	ss	pk+ct
Official-HQC	HQC-1	2,241	2,321	4,433	32	6,674
Official-HQC	HQC-3	4,514	4,602	8,978	32	13,492
Official-HQC	HQC-5	7,237	7,333	14,421	32	21,658
NSS-HQC	NSS-HQC-128	3,713	3,777	5,185	32	8,898
NSS-HQC	NSS-HQC-256	6,844	6,908	9,084	32	15,928
NSS-HQC	NSS-HQC-384	15,371	15,467	18,571	48	33,942
NSS-HQC	NSS-HQC-512	27,302	27,430	31,462	64	58,764

The NSS-HQC rows reflect the selected parameterization and the final API serialization. The 128-bit NSS-HQC set has larger communication than the official HQC-1 artifact in this direct size comparison, reflecting the cost of the selected NSS-HQC parameterization. At the higher submitted NSS-HQC levels, the communication sizes should be read together with the corresponding security targets and parameter choices rather than as a blanket dominance statement over every standard HQC parameter set.

5.7.1 Runtime Comparison on the Benchmark Platform

Table 17. Median wall-clock runtime comparison under the common benchmark harness (ms).

Scheme	Level	KeyGen	Encaps	Decaps
Official-HQC	HQC-1	0.948850	1.920750	2.945850
Official-HQC	HQC-3	2.910300	5.850900	8.953450
Official-HQC	HQC-5	7.306050	15.476150	24.574750
NSS-HQC	NSS-HQC-128	7.836450	14.280550	27.423400
NSS-HQC	NSS-HQC-256	23.998000	43.798600	76.349600
NSS-HQC	NSS-HQC-384	82.173450	150.560700	244.585850
NSS-HQC	NSS-HQC-512	198.644600	364.728050	578.171550

The measured runtime comparison is platform- and harness-dependent. It is included to provide an implementation-level reference point, while the primary communication-size comparison remains independent of the benchmark platform. On this portable artifact and benchmark harness, NSS-HQC runtime reflects a conservative implementation profile relative to the rebuilt official HQC reference libraries.

Table 18. Correctness and tamper-rejection checks in the benchmark dataset.

Parameter set	Cases	Success	Failure	Tamper trials	Tamper accepted
NSS-HQC-128	100	100	0	300	0
NSS-HQC-256	100	100	0	300	0
NSS-HQC-384	100	100	0	300	0
NSS-HQC-512	100	100	0	300	0

These tests are implementation-level checks over the deterministic benchmark dataset and tamper trials.

5.8 Memory Footprint Considerations

Memory use is dominated by ring buffers, ciphertext buffers, and the decoding workspace. Decapsulation uses additional buffers because the FO-style re-encryption check reconstructs and compares a candidate ciphertext before selecting the output key. The exact peak memory footprint depends on compiler choices, stack layout, and implementation backend. Peak-memory profiling is implementation- and platform-dependent and is therefore treated as an implementation-engineering metric rather than a scheme-level parameter in this artifact. The final benchmark focuses on correctness, tamper behavior, API sizes, and wall-clock runtime.

5.9 Summary of Performance Characteristics

The final artifact-level benchmark confirms that all four parameter sets execute successfully under the test harness, with zero correctness failures and zero accepted tampered ciphertexts in the reported trials. The measured runtime values are reported for the portable implementation. The main scheme-level communication data are given by the final API sizes, while lower-level SIMD and backend-specific optimizations remain implementation opportunities. The ciphertext

reduction remains a scheme-level size property relative to the unquantized NSS-HQC baseline at the same parameters.

6 Feature Claims

This section summarizes the distinctive features and advantages of NSS-HQC, drawing on the security analysis (Section 4) and performance evaluation (Section 5) as primary evidence.

6.1 Innovation

NSS-HQC introduces a **joint optimization framework** that unifies two independently proposed HQC improvement techniques — ciphertext compression [3] and asymmetric error rebalancing [4] — into a single coherent design with an explicit interaction analysis.

The core design idea is the exploitation of the multiplicative coupling between these two techniques through the BSC cascade formula (Eq. 66):

$$p_{\text{combined}} = p_{\text{struct}} + p_{\text{quant}} - 2p_{\text{struct}} \cdot p_{\text{quant}}.$$

This formula reveals that reducing p_{struct} (via asymmetric weights) not only lowers the structural error but also changes the allowable operating point for the quantization error p_{quant} . In the evaluated NSS-HQC parameter sets, this interaction enables a better bandwidth–DFR trade-off than applying either technique in isolation. The main contribution is to analyze these two effects in one parameter-search framework rather than as two unrelated bandwidth optimizations.

Evidence from security and performance analysis:

- Ciphertext compression alone (Bitzer et al. [3]) achieves 4–6% ciphertext reduction.
- Asymmetric rebalancing alone (Guan et al. [4]) achieves 10–12% bandwidth reduction.
- NSS-HQC’s joint optimization achieves 16–30% total ciphertext reduction relative to the unquantized NSS-HQC baseline in the evaluated parameter sets (Table 14).

In addition, NSS-HQC uses a **reliability-based errors-and-erasures RS decoding** strategy that extracts reliability information from the inner RM decoder (via the FHT reliability metric ρ_i , Eq. 13) and uses it to improve the outer RS decoder’s correction capacity (Eq. 18). While soft-decision decoding for HQC has been explored by HARE [5], the present design integrates reliability-based erasures with both ciphertext compression and asymmetric weights in the same DFR framework.

6.2 Simplicity

NSS-HQC achieves its bandwidth improvements through **minimal modifications** to the standard HQC specification [11]. The algorithmic changes are confined to three lightweight operations, none of which alter the core algebraic structure of HQC:

1. **Weight parameter change** (zero implementation cost): The ephemeral weight triple (w_{r_1}, w_{r_2}, w_e) is read from a parameter table rather than using symmetric weights. This requires changing three constants in the parameter header file; no algorithmic code is modified.
2. **Dither generation and application** (Encaps and Decaps): A pseudorandom dither vector d is generated via SHAKE256 (Eq. 29) and XORed with the ciphertext component. This

operation is localized to the ciphertext-processing path and is treated in Section 5 as part of the portable artifact runtime rather than as a separately measured primitive.

3. **Blockwise quantization** (Encaps) and **dequantization** (Decaps): Each group of mult bits is compressed to b bits via integer rounding and decompressed by the inverse map. These operations are lightweight bit-level transformations relative to the ring arithmetic and decoding components, but the final benchmark reports them only through end-to-end runtime.

What remains unchanged:

- Public key generation: the public key $pk = (\text{seed}_{\text{PKE}}, s)$ uses the same algebraic structure as standard HQC. The secret key format differs: NSS-HQC stores $sk = (\text{seed}_{\text{sk}}, \sigma, pk)$ with on-demand re-derivation of (x, y) via $I(\text{seed}_{\text{sk}})$.
- Polynomial ring structure: $R = \mathbb{F}_2[X]/(X^n - 1)$ with the same arithmetic.
- Inner code: RM(1,7) with FHT decoding, unchanged.
- Outer code: shortened RS over $\mathbb{F}_{256}$, same algebraic structure (the decoder is enhanced from hard-decision to errors-and-erasures, but the code itself is unchanged).
- FO transform structure: same $\text{FO}_m^\perp$ transform with implicit rejection.
- Hash functions: SHA3-512 and SHAKE256, same instantiation.
- Security assumptions: the public-key distribution remains the standard HQC DQCS family, while the PKE proof uses the JPCI assumption to capture the correlated encryption mask (Remark 13).

The NSS-HQC-specific implementation changes are localized to parameter handling, ciphertext compression/decompression, dither generation, and errors-and-erasures decoding support. The exact code footprint is implementation-dependent and is therefore not used as a performance metric.

6.3 Flexibility

NSS-HQC provides a **discrete, multi-dimensional parameter space** for bandwidth–computation–security trade-offs, offering greater design flexibility than the fixed standard HQC parameter table.

Three independent tuning dimensions:

1. **Quantization level** $Q \in \{2, 4, 8, 16, \dots\}$: Controls the compression aggressiveness. Higher Q reduces the quantization error p_{quant} but yields less compression. Lower Q compresses more but increases p_{quant} . The current parameter sets use $Q = 4$ (2 bits per quantized symbol), but $Q = 2$ (1 bit, maximum compression of 1/mult) or $Q = 8$ (3 bits, gentler compression) are viable alternatives for different deployment scenarios.
2. **Weight distribution** (w_{r_1}, w_{r_2}, w_e) : Controls the structural error balance. The total weight budget $w_{r_1} + w_{r_2} + w_e$ is fixed by security requirements, but the distribution can be optimized. The current parameter sets use $w_{r_1} = w_{r_2}$ (symmetric ephemeral reduction), but asymmetric configurations with $w_{r_1} \neq w_{r_2}$ are also possible and may yield marginal additional gains.
3. **Erasur threshold** $\tau \in [0, 1]$: Controls the erasure–error trade-off in RS decoding. Higher τ declares more erasures (consuming RS redundancy for erasure correction but

reducing the chance of undetected errors); lower τ declares fewer erasures (reserving RS redundancy for error correction). The optimal τ depends on the error distribution and is jointly optimized with the other parameters.

Deployment-specific configurations:

- *Bandwidth-constrained environments* (IoT, satellite): Evaluate more aggressive compression choices such as $Q = 2$, followed by a fresh DFR and security-margin search.
- *Latency-sensitive environments* (TLS handshake): Use moderate compression ($Q = 4$) with the submitted weight rebalancing for a balanced speed–bandwidth trade-off.
- *Ultra-conservative deployments*: Evaluate $Q = 8$ with minimal rebalancing for a larger correctness margin, at the cost of smaller bandwidth savings.

6.4 Compatibility

NSS-HQC maintains **structural and interface compatibility** with the standard HQC specification at the algorithmic level, enabling reuse of core building blocks in existing HQC deployments.

Component reusability:

- **Public key format:** The NSS-HQC public key $\text{pk} = (\text{seed}_{\text{PKE}}, s)$ uses the same serialization structure as standard HQC. However, because NSS-HQC may use a different ring modulus n , the byte length differs and cross-scheme key exchange is not supported. The shared format simplifies integration with certificate parsers and key-management protocols that already handle HQC keys.
- **Ring arithmetic:** All polynomial operations take place in the same ring family $R = \mathbb{F}_2[X]/(X^n - 1)$. Existing optimized implementations of polynomial multiplication (schoolbook, Karatsuba, FFT-based) can be reused without modification.
- **Error-correcting code:** The concatenated RM+RS code structure is preserved. Existing RM encoder/decoder (FHT) and RS encoder/decoder implementations are directly reusable. The only decoder modification is the addition of erasure handling to the RS decoder, which is a standard extension of the Berlekamp–Massey algorithm (Section 2.6.3).
- **Hardware accelerators:** Existing HQC hardware implementations for FPGA platforms can be adapted to NSS-HQC by adding the lightweight quantization/dequantization modules. The polynomial multiplication accelerators, which dominate the hardware area, are unchanged.

API compatibility: NSS-HQC implements the same three-function KEM API as the standard HQC specification:

- $\text{KeyGen}() \rightarrow (\text{ek}, \text{dk})$: Identical interface and semantics.
- $\text{Encaps}(\text{ek}) \rightarrow (\text{K}, \text{ct})$: Same interface; ciphertext format differs (compressed $\hat{v}$ instead of raw v).
- $\text{Decaps}(\text{dk}, \text{ct}) \rightarrow \text{K}$: Same interface; internal decompression is transparent to the caller.

6.5 Scalability

NSS-HQC scales naturally to all four required security levels (128, 256, 384, 512 bits classical) through a **uniform parameterization strategy**: only the ring dimension n , the weight parameters, and the RS code length n_1 change between security levels; the algorithmic structure remains identical.

Evidence from the security strength analysis (Section 4.5):

Table 19. Scalability summary: all four security levels use the same algorithmic framework.

Parameter set	Classical	Quantum	$T_{\text{key}}^{\text{cl}}$	$T_{\text{key}}^{\text{qu}}$	ct reduction
NSS-HQC-128	≥ 128	≥ 80	188.2	≥ 94.1	−30.0%
NSS-HQC-256	≥ 256	≥ 128	256.0	128.0	−27.1%
NSS-HQC-384	≥ 384	≥ 192	384.0	192.0	−20.6%
NSS-HQC-512	≥ 512	≥ 256	512.0	256.0	−16.6%

The ciphertext reduction (relative to the unquantized NSS-HQC baseline at the same parameters) is consistently in the range 16–30% across all security levels, demonstrating that the joint optimization framework does not lose effectiveness at higher security targets. The compression ratio on the v component is $1 - b/\text{mult} = 1 - 2/5 = 60\%$, which is a structural constant independent of n .

Evidence from the performance analysis (Section 5): The benchmarked artifact reports end-to-end portable runtime, final API sizes, and correctness/tamper behavior. The total comparison with standard HQC depends on the selected ring dimension n , serialization sizes, and implementation backend at each security level.

6.6 Comparison with Other HQC Improvement Proposals

To contextualize NSS-HQC’s contributions, Table 20 compares it against representative HQC optimization proposals cited in this report.

Table 20. NSS-HQC in the landscape of HQC optimization proposals.

Proposal	Technique	Bandwidth	Computation	Approach
Bitzer et al. [3]	Ciphertext compression	−4–6%	+<1%	Standalone
Guan et al. [4]	Asymmetric weights	−10–12%	−6–8%	Standalone
HARE [5]	Erasure decoding	−12–13%	+<2%	Standalone
Bitzer et al. [6]	GCC + reliability	−4–5%	+<1%	Standalone
Cai–Zhang [7]	GMD RS decoder	(code only)	−5%	HW-specific
Baldi et al. [8]	Structure-aware decoding	−5–8%	+<1%	Standalone
NSS-HQC	Joint compress + rebalance	−16–30%	Portable artifact measured in Sec. 5	Integrated

Key differentiators of NSS-HQC:

1. **Strong evaluated bandwidth reduction.** NSS-HQC achieves 16–30% ciphertext reduction relative to the unquantized NSS-HQC baseline at the same parameters. In the evaluated parameter sets, the combined design gives a better bandwidth–DFR operating point than treating compression and rebalancing independently.

2. **Unified DFR framework.** Many standalone proposals analyze their DFR impact in isolation. NSS-HQC provides a unified DFR model (Section 3) that jointly accounts for the structural error distribution (affected by asymmetric weights), the quantization error distribution (affected by compression parameters), and the erasure-based coding gain (affected by the reliability threshold τ). This model enables global parameter optimization rather than sequential, greedy optimization.
3. **Localized implementation changes.** Despite combining two optimization techniques, the additional quantization and dithering operations remain localized bit-processing steps around the ciphertext path. Section 5 reports their effect through end-to-end portable runtime rather than through unsupported per-primitive overhead percentages.
4. **Complementarity.** NSS-HQC is *complementary* to other proposals rather than competing with them. The GCC framework [6], the GMD decoder [7], the structure-aware decoding [8], and the closed-form DFR model [10] can all potentially be integrated with NSS-HQC’s joint framework in future work, promising further improvements.

6.7 Summary

NSS-HQC offers the following combination of properties in the evaluated parameter sets (Table 21).

Table 21. Summary of NSS-HQC properties.

Property	NSS-HQC
Ciphertext reduction vs. unquantized NSS-HQC baseline	16–30%
Benchmark scope	Portable runtime, API sizes, correctness, and tamper checks
Security assumption	DQCSD-family / JPCI (HQC-style)
Public key format change	None
Core modifications vs. NIST HQC	3 (Sec. 1.3, Remark)
IND-CCA2 proof in QROM	Yes (Theorem 18)
Supported security levels	128 / 256 / 384 / 512 bit
Quantum security compliance	≥ 80 / 128 / 192 / 256 bit
Encapsulated key κ	$\geq$ classical target
Implementation delta	Localized compression, dither, and erasure-decoding support

In summary, NSS-HQC demonstrates that significant bandwidth improvements for code-based KEMs are achievable through *principled integration* of complementary optimization techniques, while keeping the security assumptions, correctness margins, implementation changes, and platform requirements explicit.

References

- [1] NIST, “Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process,” NIST IR 8545, March 11, 2025. <https://doi.org/10.6028/NIST.IR.8545>.
- [2] E. Dominguez Fiallo, P. Freyre Arrozarena, L. R. Piñeiro, “Decoding Quasi-Cyclic Codes is NP-complete,” IACR ePrint 2023/1064, 2023.
- [3] S. Bitzer, J.-C. Deneuville, E. Munisamy, B. Purtili, S. Ritterhoff, A. Wachter-Zeh, “HQC Beyond the Standard: Ciphertext Compression and Refined DFR Analysis,” IACR ePrint 2025/2259, EUROCRYPT 2026.
- [4] C. Guan, L. Luo, H. Jiang, J. Hou, T. Yu, H. Wang, K. Li, L. Qu, “Compact HQC with new (un)balance,” IACR ePrint 2026/461, 2026.
- [5] T. Wang, Q. Teng, A. Wang, J. Zhang, B. Pang, C. Zhao, S. Hu, X. Wang, “HARE: Compact HQC via Distance-Informed Erasure Decoding,” IACR ePrint 2026/544, 2026.
- [6] S. Bitzer, B. Purtili, A. Wachter-Zeh, “Improved Codes and Decoders for HQC,” IACR ePrint 2026/656, 2026.
- [7] J. Cai, X. Zhang, “HQC Post-Quantum Cryptography Decryption with Generalized Minimum-Distance Reed-Solomon Decoder,” arXiv:2603.20156, 2026.
- [8] M. Baldi, S. Bitzer, N. Lilla, P. Santini, “HQC Beyond the BSC: Towards Error Structure-Aware Decoding,” IACR ePrint 2025/648, EUROCRYPT 2026.
- [9] C. Nugier, J.-C. Deneuville, “Better Codes for the HQC Cryptosystem,” IACR ePrint 2025/114, 2025.
- [10] A. Annechini, A. Barengi, G. Pelosi, “On the Decoding Failure Rate of HQC,” IACR ePrint 2026/795, 2026.
- [11] C. Aguilar Melchor, N. Aragon, S. Bettaieb, L. Bidoux, O. Blazy, J.-C. Deneuville, P. Gaborit, E. Persichetti, G. Zémor, J. Bos, A. Dion, J. Lacan, J.-M. Robert, P. Véron, “Hamming Quasi-Cyclic (HQC),” HQC specification, Aug. 22, 2025. <https://pqc-hqc.org>.
- [12] D. Hofheinz, K. Hövelmanns, E. Kiltz, “A Modular Analysis of the Fujisaki–Okamoto Transformation,” TCC 2017, LNCS 10677, pp. 341–371.
- [13] A. Esser, E. Bellini, “Syndrome Decoding Estimator,” PKC 2022, LNCS 13177, pp. 112–141.
- [14] S. Narisada, H. Okada, Y. Aikawa, K. Fukushima, “Refined Analysis of the Concrete Hardness of the Quasi-Cyclic Syndrome Decoding,” IWSEC 2025. https://doi.org/10.1007/978-981-95-4674-9_7.
- [15] D.J. Bernstein, “Grover vs. McEliece,” PQCrypto 2010, LNCS 6061, pp. 73–80.